



Kristu Jayanti College

AUTONOMOUS

Bengaluru

Reaccredited 'A++' Grade by NAAC | Affiliated to Bengaluru North University

PROJECT REPORT ON

“KJC VOX – A Student Feedback System”

Submitted in partial fulfillment of the requirements for Enterprise
Computing Project Lab for 3rd Semester

Master of Computer Applications (MCA)

Submitted by

Alluri Abishek Kumar 24MCAB07

S Dinesh 24MCAB17

Under the guidance of

Dr. Princess Maria John



Kristu Jayanti College

AUTONOMOUS

Bengaluru

Reaccredited 'A++' Grade by NAAC | Affiliated to Bengaluru North University

CERTIFICATE

This is to certify that the project titled “***KJC VOX – A Student Feedback System***” has been satisfactorily completed by **Mr. Alluri Abishek Kumar** with **24MCAB07**, in partial fulfillment of the requirements for ***Enterprise Computing Project*** with course code “**MCC2P2B31**”, for the 3rd Semester MCA course during the academic semester from July 2025 to November 2025 as prescribed by Bangalore North University.

Faculty In-charge

Head of the Department

Valued by

Examiner 1: _____

Date: _____

Examiner 2: _____

Centre: Kristu Jayanti College

ACKNOWLEDGEMENT

First of all, we would like to thank the God Almighty for all the blessings he has showered on us. Our spiritual quotient gave us more strength and motivation that helped immensely.

We would like to thank **Rev. Fr. Dr. Augustine George**, our Principal, for providing us his constant guidance and support. I would also like to thank **Rev. Fr. Dr. Lijo P Thomas**, our Vice-Principal, for providing us with the best facilities.

We are extremely thankful to **Dr. Muruganathan A**, Head, Department of Computer Science (PG) for giving us the essential support in the form of allocating comfortable project hours and necessary software resources.

We would like to extend our heartfelt thanks to **Dr. Princess Maria John**, our project guide for providing us the necessary details related to project development and process identification enabling us to finish the project within the stipulated time.

We thank all other faculty members who helped us a lot in completing this project.

We thank our class mates, who have pointed out errors and guided us a lot and we thank each and every one who has helped us.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Abstract	vi
	List of Tables	vii
	List of Figures	viii
1	INTRODUCTION	
	1.1 Problem Statement	1
	1.2 Project Description	1
2	SYSTEM STUDY	
	2.1 Existing System	4
	2.2 Proposed System	5
	2.3 Use Case Diagram	6
	2.4 Sequential Diagram	8
	2.5 Activity Diagram	10
3	SYSTEM CONFIGURATION	
	3.1 Hardware Configuration	11
	3.2 Software Configuration	12
4	SOFTWARE DESCRIPTION	
	4.1 Overview Front End	13
	4.2 Overview Back End	14
5	SYSTEM DESIGN	
	5.1 Architectural	16
	5.2 Input Design	17

CHAPTER NO	TITLE	PAGE NO
	5.3 Output Design	19
	5.4 Database Design	22
6	SOURCE CODE	27
7	SYSTEM TESTING	60
8	SYSTEM DEPLOYMENT	67
9	CONCLUSION AND FUTURE ENHANCEMENT	70
10	APPENDICES	
	10.1 Screenshots	73
11	REFERENCES / BIBLIOGRAPHY	77

ABSTRACT

In modern academic institutions, effective communication between students and faculty is essential for improving the quality of education. However, traditional feedback collection methods — such as manual forms or open surveys — often fail to ensure privacy, authenticity, and transparency. Students may hesitate to express honest opinions due to fear of identification, while faculty and administrators lack efficient tools to analyze and act upon the feedback data. These limitations highlight the need for a secure, anonymous, and data-driven system that bridges the communication gap and promotes continuous institutional improvement.

The KJC Vox Feedback System is a web-based platform designed to provide students with a safe and confidential environment to share feedback about faculty and courses. Built with Angular for the frontend, Java (Spring Boot) for the backend, and MongoDB Atlas for cloud-based storage, the system ensures scalability, security, and performance. It implements role-based access control for three types of users — Students, Faculty, and Administrators — each with tailored functionalities. Students can submit feedback anonymously, faculty can view only aggregated ratings and comments, and administrators can manage users, subjects, and departments while monitoring overall system performance. Additionally, the system integrates JavaMail API for OTP-based email verification and password recovery, ensuring secure authentication and account management.

By combining secure authentication, anonymized feedback submission, and real-time analytics, KJC Vox transforms the traditional feedback process into a transparent, efficient, and insightful digital platform. It empowers students to voice their opinions freely, helps faculty identify areas for self-improvement, and enables administrators to make data-backed decisions for academic excellence. Ultimately, the KJC Vox Feedback System serves as a modern step toward building a feedback culture rooted in trust, integrity, and continuous development within educational institutions.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.1	Use Case Diagram	07
2.2	Sequential Diagram	09
2.3	Activity Diagram	10
5.1	Architectural Diagram	16
5.2	Registration Page	18
5.3	Login Page	19
5.4	Student Dashboard Page	20
5.5	Assigned Courses and Faculty Page	21
5.6	Feedback Form Page	22
10.1	Registration Page	73
10.2	Login Page	73
10.3	Role Selection Page	74
10.4	Student Dashboard Page	74
10.5	Student Course view Page	75
10.6	Feedback form Page	75
10.7	Add Department Page	76
10.8	Add Notice Page	76

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Hardware Configuration	11
3.2	Software Configuration	12
4.1	Technologies Used in Front-End	13
4.2	Technologies Used in Back-End	15
5.1	MCA Collection	23
5.2	departments Collection	24
5.3	faculty Collection	24
5.4	Feedbacks collection	25
5.5	student-login Collection	26
5.6	subjects_assigned Collection	26
7.1	User Registration Form	63
7.2	Login Form	63
7.3	Feedback Form	64
7.4	Dashboard & Notice Module	65
7.5	Data Consistency & Sync	66

1. INTRODUCTION

1.1 Problem Statement

In academic environments, student feedback plays a vital role in assessing the effectiveness of teaching methods and improving the overall learning experience. However, traditional feedback collection processes—such as paper forms or open online surveys—often fail to ensure anonymity, data privacy, and authenticity. As a result, students may hesitate to express their honest opinions due to fear of being identified or judged, leading to incomplete or biased feedback.

Moreover, faculty members and administrators lack an efficient and centralized platform to manage, analyze, and interpret feedback data. Without a structured system, it becomes challenging to monitor performance trends, identify areas for improvement, or take timely corrective measures.

The problem, therefore, is to develop a secure, anonymous, and role-based feedback management system that allows students to share their views freely, while providing faculty with summarized insights and enabling administrators to manage the entire process transparently. The system should implement robust authentication, data encryption, and access control mechanisms to protect sensitive information and maintain data integrity.

1.2 Project Description

The KJC Vox Feedback System is a web-based application developed to provide a secure, structured, and anonymous platform for students to share feedback about their faculty and courses. It promotes transparency and open communication within academic institutions by ensuring complete anonymity for students while offering faculty and administrators valuable insights through summarized reports and analytics. The system uses role-based authentication, where students can submit feedback, faculty can view aggregated responses, and administrators can manage users and monitor data. Built with Angular for the frontend, Java for backend logic, and

MongoDB Atlas for cloud-based data storage, the platform ensures scalability, data integrity, and privacy. Additionally, JavaMail API is integrated for OTP-based email verification and password recovery, enhancing user security. Overall, KJC Vox serves as a reliable, user-friendly, and efficient solution that simplifies the feedback process, maintains confidentiality, and helps institutions improve teaching quality through data-driven decisions.

Module Description

The KJC Vox Feedback System is divided into several main modules, each handling specific functionality:

1. User Authentication & Management Module

- Handles secure login and registration for Students, Faculty, and Administrators.
- Uses OTP-based email verification through the JavaMail API.
- Implements hashed passwords and role-based access control to ensure security and privacy.

2. Student Module

- Allows students to submit anonymous feedback about faculty and courses.
- Provides rating-based and comment-based input options for detailed feedback.
- Ensures that no identifying student data is linked to the submitted feedback.

3. Faculty Module

- Enables faculty members to view aggregated feedback reports and performance analytics.
- Displays statistical summaries and comment insights without revealing student identities.
- Helps faculty improve their teaching methods based on structured feedback data.

4. Admin Module

- Provides administrators with full control to manage users and feedback records.
- Allows adding, editing, or deleting user accounts and monitoring overall system activity.
- Generates comprehensive reports for institutional evaluation and improvement.

5. **Feedback Management Module**

- Stores all feedback securely in **MongoDB Atlas**, maintaining data integrity and confidentiality.
- Links each feedback record with the respective student, faculty, and course entities.

6. **Email Integration Module (JavaMail API)**

- Sends OTP for email verification and password reset processes..
- Ensures secure user authentication and account recovery.

2. SYSTEM STUDY

2.1 Existing System

In many educational institutions today, student feedback is still collected through traditional manual or semi-digital methods. These include printed feedback forms, basic Google Forms, or simple web pages that lack authentication, validation, and anonymity assurance. Such systems are often static and do not adapt to different roles like students, faculty, or administrators. The absence of real-time validation, secure storage, and analytics makes it difficult for institutions to make effective use of the collected data.

Additionally, existing feedback systems fail to maintain complete confidentiality. In many cases, students are reluctant to provide honest feedback due to fear of being identified. There is no mechanism to ensure that submissions remain anonymous or to detect inappropriate language or misuse in feedback entries. Faculty, on the other hand, receive feedback data in raw or unfiltered form, making it hard to analyze or identify trends.

Most legacy systems lack role-based access control — meaning all users often share the same level of access. Administrators cannot effectively manage users, update subjects, or generate analytical insights without manual intervention. Furthermore, data visualization and reporting are usually limited to static spreadsheets, which fail to provide real-time insights or actionable summaries. The limitations of the existing systems can be summarized as follows:

- Lack of role-based and secure login mechanisms.
- No assurance of anonymity in student feedback.
- Absence of data validation and misuse detection.
- Manual or semi-automated reporting processes.
- Limited feedback aggregation and analytics.
- Static interfaces with minimal interactivity or scalability.

These drawbacks highlight the need for a modern, secure, and intelligent feedback management system that ensures anonymity, accuracy, and real-time analytics while maintaining ease of use for students, faculty, and administrators.

2.2 Proposed System

The proposed system — KJC Vox: Anonymous Feedback Management Platform — aims to address the shortcomings of traditional feedback systems by implementing a secure, role-based, and intelligent web solution. It allows students to submit feedback anonymously, faculty to view summarized results, and administrators to manage users, departments, and announcements efficiently.

The system leverages modern technologies such as Angular (Frontend), Spring Boot (Backend), and MongoDB Atlas (Database) to deliver a scalable, reliable, and responsive feedback platform. Authentication is implemented using JWT-based security, ensuring that each user role (Student, Faculty, Admin) has controlled access to specific operations.

Students can log in with their credentials, verify via OTP (One-Time Password), and access their personalized dashboard. They can view assigned subjects, select a faculty member, and submit feedback securely. All responses are stored in MongoDB, preserving anonymity and integrity. The system also validates input to prevent misuse, such as inappropriate or spam content.

Faculty members can log in to view aggregated feedback reports filtered by course, semester, or department. These reports display ratings, comments, and performance insights — all anonymized to protect student identity. This encourages transparent self-improvement without bias.

Administrators can perform CRUD operations such as managing user accounts, assigning subjects, and posting notices or announcements. All operations are securely logged, ensuring data accountability and traceability.

The proposed system offers the following advantages:

- Secure role-based login using JWT authentication.
- Anonymous feedback submission with misuse detection.
- Real-time data visualization and reporting.
- Easy subject, faculty, and notice management for administrators.
- Scalable architecture using Spring Boot and MongoDB Atlas.
- Improved student participation through a modern, user-friendly UI.
- Multi-role access (Student, Faculty, Admin) with personalized dashboards.

Overall, KJC Vox modernizes the feedback process by combining security, simplicity, and transparency. It promotes honest student participation, enables faculty reflection, and helps administrators make data-driven decisions to enhance academic quality.

2.3 Use Case Diagram

The Use Case Diagram represents the functional interactions among the three primary actors — Student, Faculty, and Admin — and the KJC Vox Feedback System.

It outlines how each actor communicates with the system to perform their respective tasks:

- Students can register, log in, and submit anonymous feedback.
- Faculty members can log in and view summarized feedback reports.
- Admins can manage users, assign subjects, post notices, and oversee operations.

This diagram provides a clear overview of the system's core functionalities and the boundaries of each user role, serving as the foundation for defining and validating system requirements.

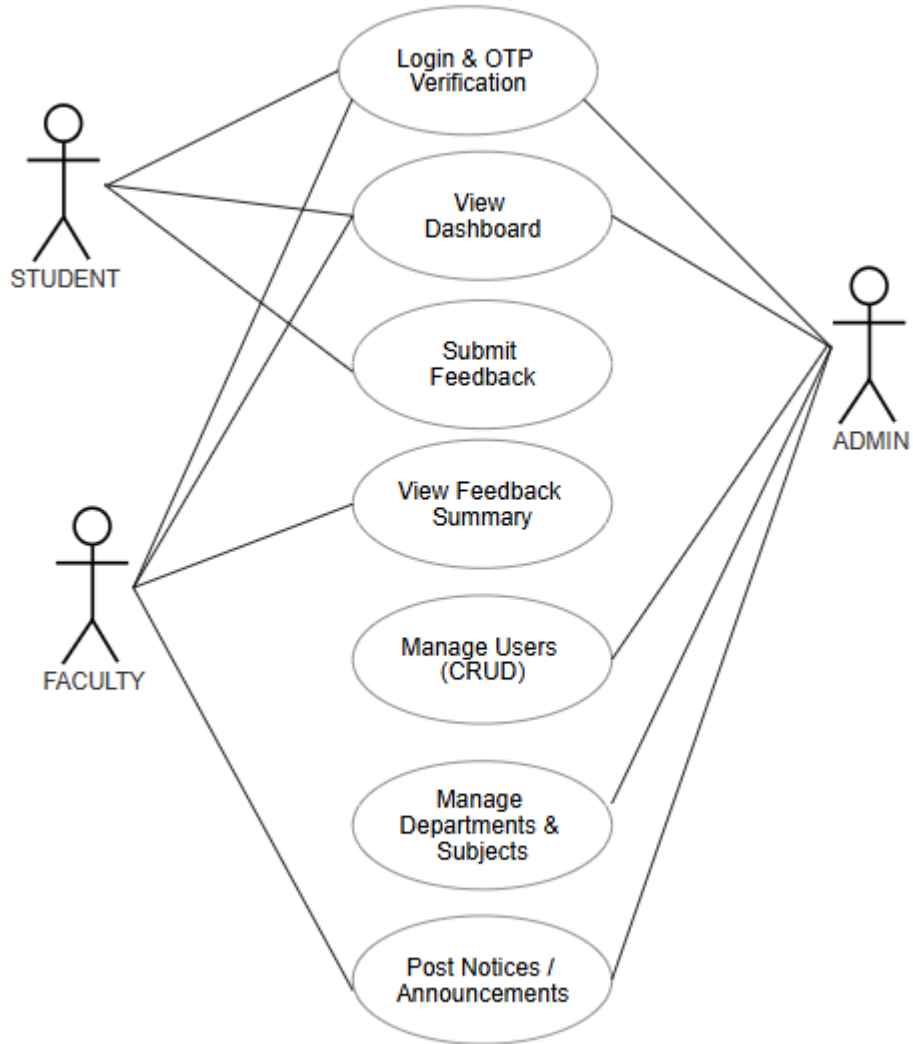


Figure 2.1: Use Case Diagram

2.4 Sequential Diagram

The Sequence Diagram illustrates the flow of operations and interactions between system components during major processes such as login, feedback submission, and report generation. It shows the chronological exchange of messages among the Actors (Student, Faculty, Admin) and System Components (Frontend, Backend, Database, and Email Service).

Typical flows include:

- The student logs in → Backend verifies credentials → OTP sent via Email → OTP verified → Access granted.
- The student submits feedback → Backend stores data in MongoDB → Success response.
- The faculty requests feedback report → Backend retrieves aggregated data → Displayed on the faculty dashboard.
- The admin performs CRUD actions for user or subject management.

This diagram provides a compact view of system behavior across all roles, ensuring all interactions are logically consistent and traceable.

Refer to Figure 2.2 – Sequence Diagram.

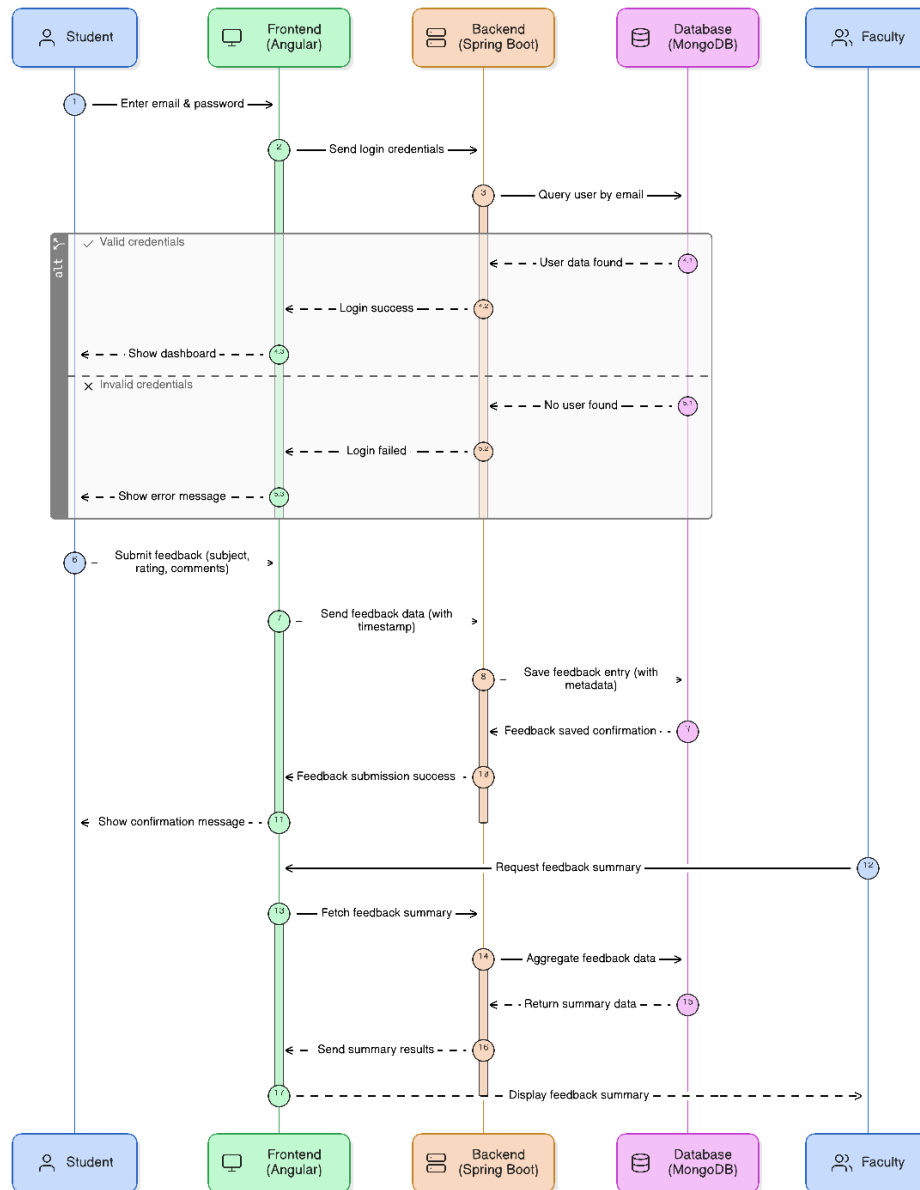


Figure 2.2: Sequential Diagram

2.5 Activity Diagram

The Activity Diagram presents the step-by-step workflow of user activities within the KJC Vox platform. It visually captures the flow of control from login and verification to feedback submission and viewing. The process begins with the student logging in or registering, proceeds to subject selection, and ends with anonymous feedback submission.

The diagram also outlines faculty activities such as logging in and viewing summarized reports, and admin activities such as managing subjects or notices. Decision nodes are used to handle validation checks and error responses during login or submission.

By illustrating parallel and conditional paths, this diagram helps visualize how different user roles navigate through the system's operations in a coordinated and secure manner.

Refer to Figure 2.3 – Activity Diagram.

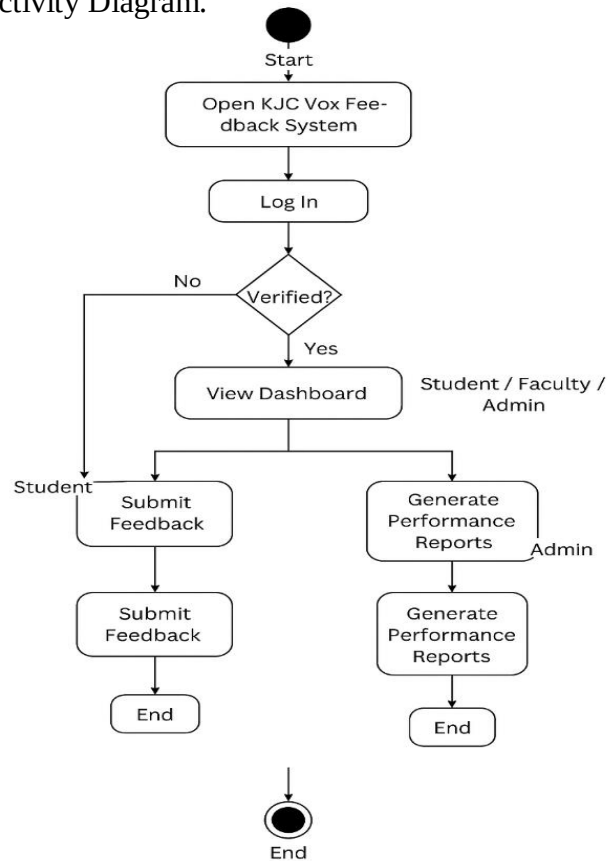


Figure 2.3: Activity Diagram

3. SYSTEM CONFIGURATION

3.1 Hardware Configuration

The hardware configuration defines the physical resources required for developing, testing, and deploying the KJC Vox Feedback System. Since the system is web-based and built using modern frameworks, it can operate efficiently on a standard computing setup. Refer to Table 3.1.

Table 3.1: Hardware Configuration

Component	Specification / Description
Processor	Intel Core i5 or equivalent (Minimum 2.4 GHz)
RAM	8 GB or higher (Recommended: 16 GB for multi-user testing and database operations)
Hard Disk	Minimum 500 GB HDD or 256 GB SSD
Monitor	15.6" or larger display with at least 1080p resolution
Input Devices	Standard Keyboard and Mouse
Network	Stable broadband or campus Wi-Fi connection
Audio Devices	Speakers / Headphones (for email notification testing, if applicable)
Operating System Support	Windows 10 / 11, Linux, or macOS

This configuration ensures smooth development, testing, and deployment of the KJC Vox Feedback System, supporting efficient server-side operations, database access, and user interface rendering across all roles — Student, Faculty, and Admin.

3.2 Software Configuration

The software configuration outlines the tools, frameworks, and technologies used to build and run the KJC Vox Feedback System. The system integrates frontend, backend, and database technologies to ensure a secure, responsive, and scalable web application. Refer to Table 3.2.

Table 3.2: Software Configuration

Category	Software / Technology Used	Description
Operating System	Windows 10 / 11	Main environment for system development and testing
Frontend	Angular, TypeScript, CSS, HTML	Used for building responsive dashboards for Students, Faculty, and Admin
Backend	Java (Spring Boot / Servlets)	Handles authentication, feedback management, and data processing
Database	MongoDB Atlas (Cloud NoSQL)	Stores user, subject, and feedback data securely with high scalability
Email Integration	JavaMail API	Used for OTP verification and password recovery functionality
Development IDE	IntelliJ IDEA / VS Code	For coding, debugging, and project management
Version Control	Git / GitHub	For version management and collaborative development
Testing Tools	Postman, JUnit	Used for API and backend testing
Web Browser	Google Chrome / Edge / Firefox	For running and testing the web-based interface

This configuration provides a stable and secure environment for implementing all system features, ensuring high performance, modularity, and ease of maintenance.

4. SOFTWARE DESCRIPTION

4.1 Overview – Front End

The front end of the KJC Vox Feedback System is designed to provide a clean, intuitive, and responsive user interface for students, faculty, and administrators. It acts as the presentation layer that facilitates interaction between users and the backend services through RESTful APIs. Each user role is provided with a customized dashboard and functionalities tailored to their responsibilities.

Developed using Angular, the frontend ensures modular, dynamic, and responsive design, adapting seamlessly to different screen sizes and devices. The use of TypeScript enhances reliability through type safety, while CSS3 and HTML5 provide elegant styling and structure. The interface focuses on clarity, accessibility, and performance, ensuring smooth navigation for users at every stage — from login and verification to feedback submission and dashboard access.

Key Features of Front-End System:

- **Developed Using Angular:** Component-based architecture for modular and scalable UI.
- **Responsive Layout:** Designed to adapt across desktops, tablets, and mobile devices using responsive CSS.
- **Role-Based Dashboards:** Separate interfaces for Students, Faculty, and Admins to ensure clear workflow separation.
- **Real-Time Communication:** Uses REST APIs for instant data retrieval and updates.
- **Secure Forms:** Input validations and restricted access ensure data privacy and prevent misuse.

Table 4.1: Technologies Used in Front-End

Technology	Purpose
Angular	Framework for building modular and dynamic user interfaces
TypeScript	Ensures strong typing and better code maintainability

HTML5 & CSS3	Provides structure and styling for all pages
JavaScript	Handles client-side scripting and event interactions
Bootstrap / Tailwind CSS	Used for responsive design and consistent layout styling
RESTful API Integration	Connects the frontend with backend services securely
Visual Studio Code / IntelliJ	IDE used for frontend development and debugging

The frontend of KJC Vox acts as the interactive bridge between users and the backend logic, ensuring smooth feedback submission, secure communication, and real-time data synchronization across all dashboards. Refer to Table 4.1 for the technologies used in the frontend.

4.2 Overview Back End

The backend of the KJC Vox Feedback System serves as the core logic and processing layer that manages data flow, handles authentication, and ensures secure communication between the user interface and the database. Developed using Java, the backend is designed with a modular architecture that supports role-based access control, secure feedback management, and real-time data updates.

The backend manages all major operations such as user authentication, OTP verification via email, feedback submission, report generation, and system administration. It ensures that feedback data remains anonymous and securely stored in the MongoDB Atlas database. The backend also integrates JavaMail API for sending OTPs and password reset emails, enhancing system security and user convenience.

Key Features of Back-End System:

- **Role-Based Access Control:** Secure authentication and authorization for Students, Faculty, and Admins.
- **OTP Verification:** Uses JavaMail API to verify users via email during registration and password recovery.
- **Feedback Management:** Collects, stores, and aggregates feedback while maintaining anonymity.
- **Database Connectivity:** Connects seamlessly to MongoDB Atlas for efficient data storage and retrieval.

- **Secure APIs:** Implements RESTful endpoints with encrypted communication and token-based access.
- **Error Logging and Monitoring:** Tracks system activity to ensure reliability and easy debugging.
- **Scalable Architecture:** Supports future expansion, analytics integration, and cloud deployment.

Table 4.2: Technologies Used in Back-End

Technology	Purpose
Java (Spring Boot / Servlets)	Core backend framework for handling logic, APIs, and data flow
MongoDB Atlas	Cloud-based NoSQL database for storing user and feedback data
JavaMail API	Used for OTP generation, verification, and password reset emails
RESTful APIs	Enables communication between frontend and backend
JSON	Data format used for client-server communication
IntelliJ IDEA	Integrated Development Environment used for coding and debugging
Git / GitHub	Version control and collaborative development
Postman	Tool for testing and validating REST APIs

The backend of KJC Vox acts as the engine of the application, ensuring data consistency, security, and seamless communication between the frontend interface and the database. It handles user authentication, feedback aggregation, and system management efficiently, enabling a smooth and secure feedback experience across all user roles.

Refer to Table 4.2 for the technologies used in the backend.

5. SYSTEM DESIGN

5.1 Architectural Diagram

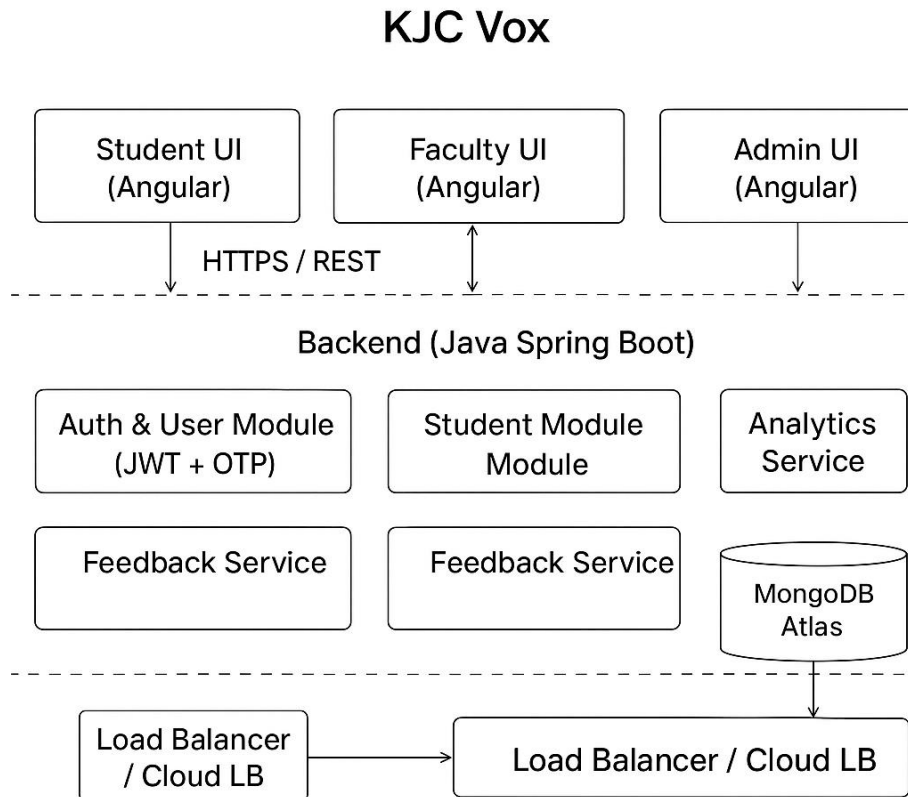


Figure 5.1 : Architectural Diagram

The KJC Vox Feedback System is designed with a three-tier architecture to ensure scalability, data security, and efficient communication among its components. The system is divided into the Presentation Layer, Application Layer, and Data Layer, each responsible for specific roles in the overall workflow.

At the Presentation Layer, the system offers three distinct user interfaces — Student, Faculty, and Admin — all developed using Angular, TypeScript, and CSS for a responsive and interactive experience. Each interface communicates securely with the backend through RESTful APIs over HTTPS, ensuring smooth and protected data transfer.

The Application Layer forms the core of the system, implemented using Java (Spring Boot). It consists of several key modules: the Authentication and User Management Module for role-based login and OTP verification, the Student Module for feedback submission, the Faculty Module for viewing summarized reports, the Admin Module for managing data and users, and the Feedback & Analytics Services that handle aggregation and report generation. Additionally, the JavaMail API integrates with this layer to manage OTP and password reset functionalities.

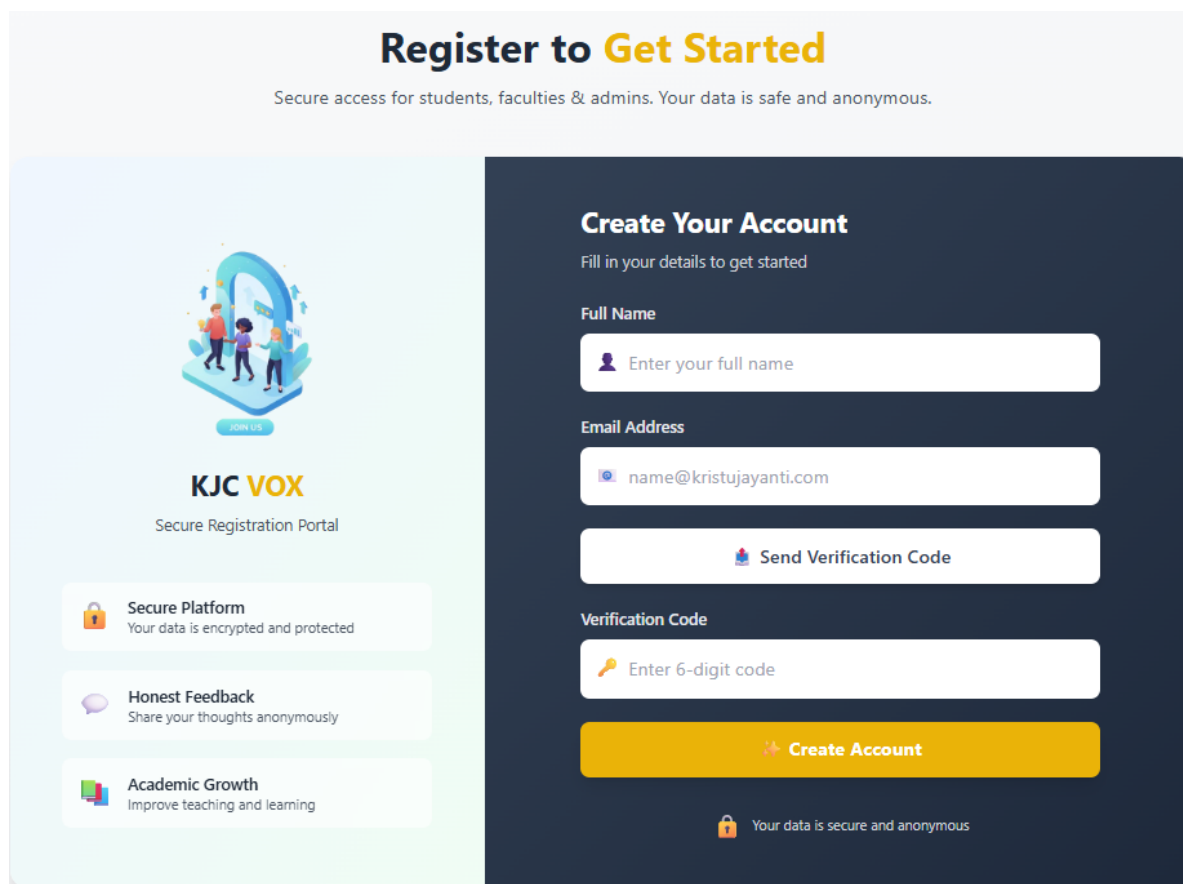
The Data Layer utilizes MongoDB Atlas, a cloud-based NoSQL database, to store all application data securely. This layer maintains relationships between users, courses, and feedback records while ensuring data integrity and privacy. Together, these layers provide a modular, secure, and maintainable architecture, making KJC Vox a reliable platform for transparent and anonymous academic feedback management.

5.2 Input Design

5.2.1 Registration Page

The Registration Page of the KJC Vox Feedback System allows new users—students, faculty, and administrators—to register securely using their institutional credentials. Users are required to enter details such as name, email ID, department, and role, after which an OTP (One-Time Password) is automatically sent to the registered email for verification through the JavaMail API.

The page features a clean and minimal layout that emphasizes simplicity, accessibility, and ease of use. Proper form validations ensure that all mandatory fields are filled and email formats are correct before submission. Once the verification is complete, users can proceed to log in and access their respective dashboards. Refer to Figure 5.2 for the Registration Page.



The screenshot displays the 'Register to Get Started' page for KJC VOX. The header features the title 'Register to Get Started' in bold, with a subtitle 'Secure access for students, faculties & admins. Your data is safe and anonymous.' Below this, the KJC VOX logo is shown with the tagline 'Secure Registration Portal'. The page is divided into two main sections. The left section, on a light blue background, lists three benefits: 'Secure Platform' (data encrypted and protected), 'Honest Feedback' (anonymous sharing), and 'Academic Growth' (improved teaching and learning). The right section, on a dark blue background, is titled 'Create Your Account' and prompts users to 'Fill in your details to get started'. It contains input fields for 'Full Name' and 'Email Address' (pre-filled with 'name@kristujayanti.com'), a 'Send Verification Code' button, a 'Verification Code' field (prompting for a 6-digit code), and a large yellow 'Create Account' button. A footer note states 'Your data is secure and anonymous' with a lock icon.

Register to Get Started

Secure access for students, faculties & admins. Your data is safe and anonymous.

KJC VOX
Secure Registration Portal

- Secure Platform**
Your data is encrypted and protected
- Honest Feedback**
Share your thoughts anonymously
- Academic Growth**
Improve teaching and learning

Create Your Account

Fill in your details to get started

Full Name
Enter your full name

Email Address
name@kristujayanti.com

Send Verification Code

Verification Code
Enter 6-digit code

Create Account

Your data is secure and anonymous

Figure 5.2: Registration Page

5.2.2 Login Page

The Login Page serves as the gateway for registered users to access their role-specific dashboards. It requires users to enter their registered email ID and password, which are validated through the backend authentication module. For first-time users or those who have forgotten their passwords, the system provides an OTP-based password recovery option, ensuring account security and smooth access.

The interface maintains consistency with the registration page in terms of color theme, typography, and layout. It offers a seamless, user-friendly experience with clear navigation, error prompts for invalid credentials, and an option to remember login details for convenience

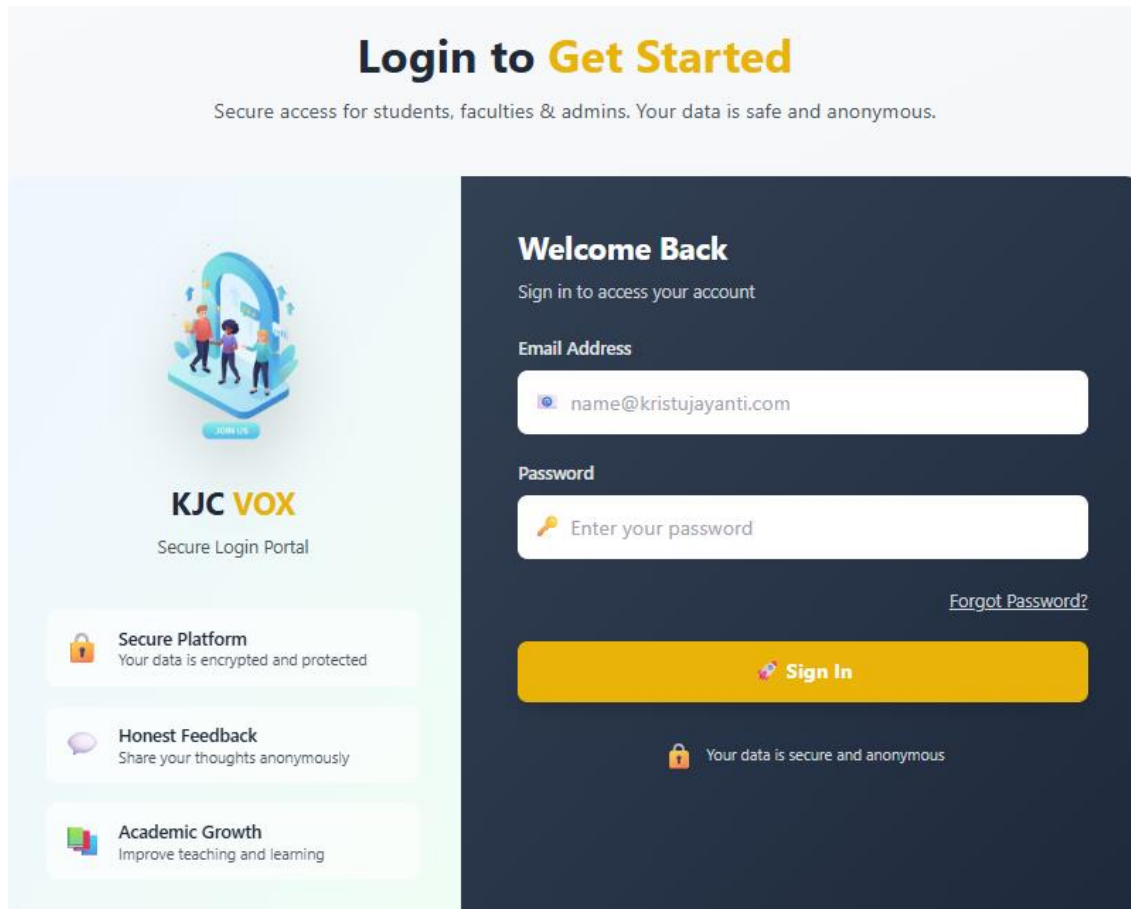


Figure 5.3: Login Page

5.3 Output Design

5.3.1 Student Dashboard Page

The **Student Dashboard** is the primary interface displayed after a student successfully logs into the KJC Vox Feedback System. It serves as a personalized space where students can view **announcements**, **upcoming assignments**, and navigate to various sections such as **All Subjects**, **My Feedback**, and **Report Issue**.

The dashboard also features motivational quotes and reminders to encourage students to participate in the feedback process. A clean and modern design ensures smooth navigation and readability, allowing students to access all system features with minimal effort. Refer to **Figure 5.4** for the Student Dashboard.

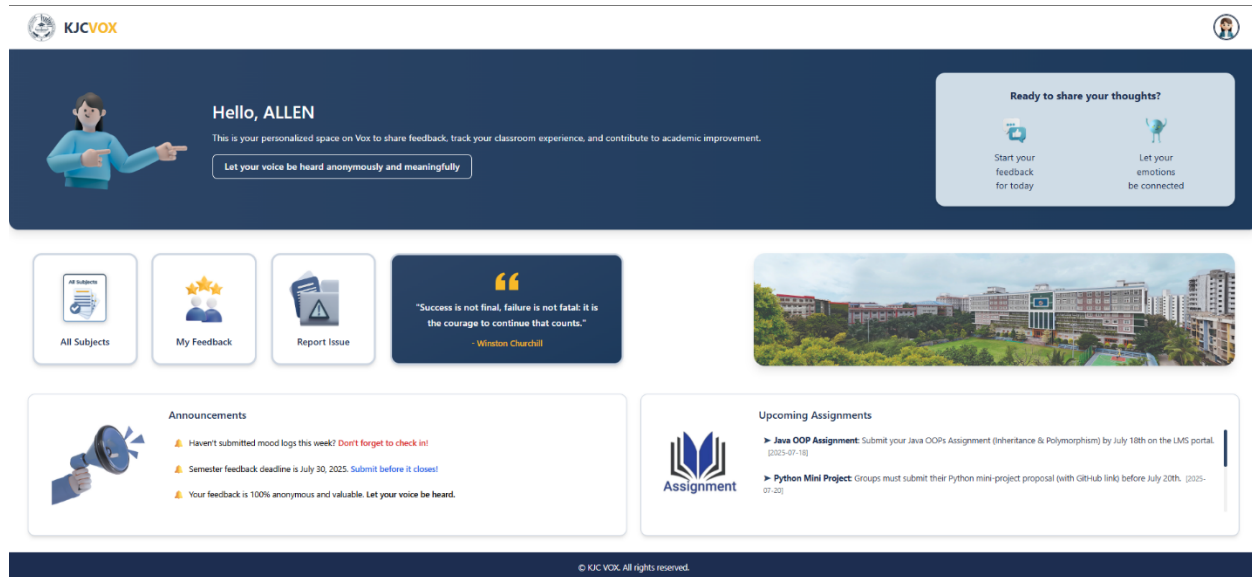


Figure 5.4: Student Dashboard Page

5.3.2 Assigned Courses and Faculty Page

This page displays the list of subjects assigned to the student for the current semester, along with corresponding faculty names and course codes. Each subject row provides a “Give Feedback” button, allowing students to rate and share their experience with respective faculty members.

The design ensures simplicity and clarity, making it easy for students to identify subjects and submit feedback efficiently. It reflects real-time course data fetched from the backend, ensuring accuracy and relevance. Refer to Figure 5.5 for the Assigned Courses and Faculty Page.

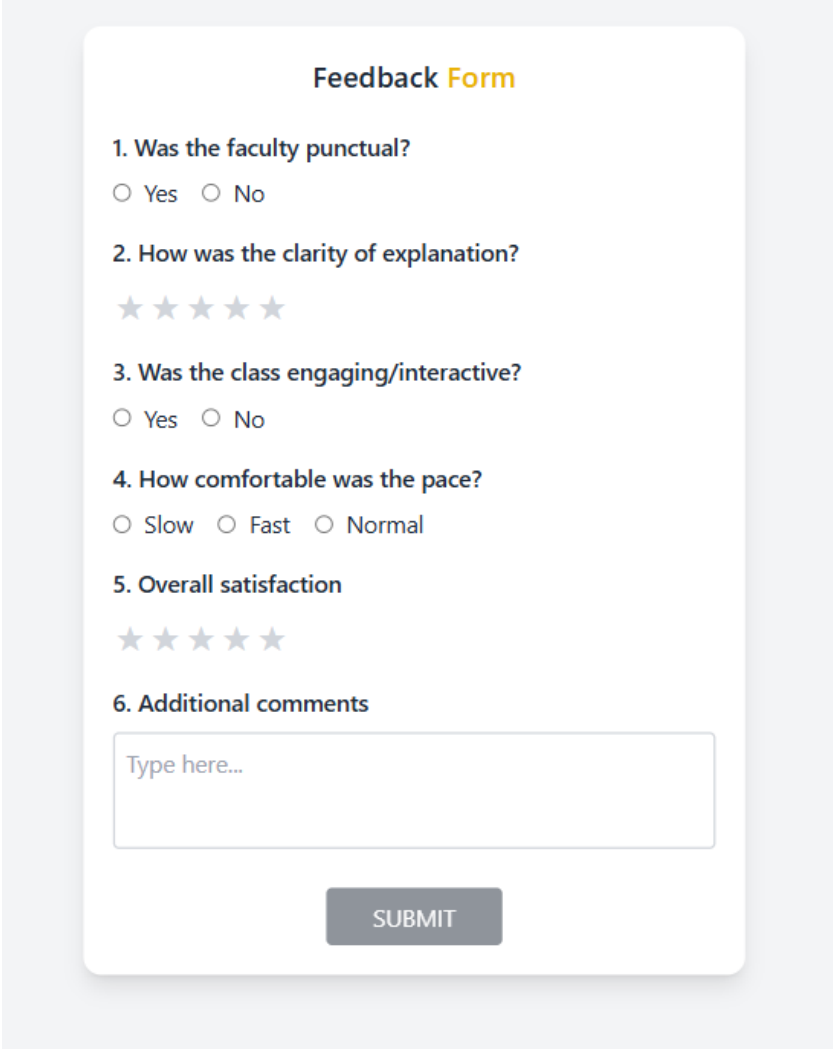


Figure 5.5: Assigned Courses and Faculty Page

5.3.3 Feedback Form Page

The Feedback Form page enables students to submit structured feedback for each subject or faculty. The form includes both rating-based questions (1–5 scale) and objective questions (Yes/No or multiple-choice), along with an optional text box for additional comments.

Questions are designed to assess various aspects of teaching quality — such as clarity of explanation, engagement level, punctuality, and overall satisfaction. Once the feedback is submitted, it is securely stored in the MongoDB database without revealing the student's identity, ensuring complete anonymity. Refer to Figure 5.6 for the Feedback Form Page.

A screenshot of a feedback form titled "Feedback Form" in bold black text with "Form" in orange. The form contains six questions. Questions 1, 3, and 4 use radio buttons for Yes/No or Slow/Fast/Normal. Questions 2 and 5 use five-star rating systems. Question 6 is an open-ended text field. A grey "SUBMIT" button is at the bottom.

Feedback Form

1. Was the faculty punctual?
☐ Yes ☐ No

2. How was the clarity of explanation?
★ ★ ★ ★ ★

3. Was the class engaging/interactive?
☐ Yes ☐ No

4. How comfortable was the pace?
☐ Slow ☐ Fast ☐ Normal

5. Overall satisfaction
★ ★ ★ ★ ★

6. Additional comments

SUBMIT

Figure 5.6: Feedback Form Page

5.4 Database Design

The database design of **KJC Vox Portal** focuses on securely storing student, faculty, and department-related data, along with course assignments, feedback records, and announcements. Since this system is built on **MongoDB**, it uses a **document-oriented structure**, allowing flexibility in managing nested and hierarchical data such as subjects, assigned teachers, and student feedback forms.

Each collection in the database is structured to support different modules of the portal — such as authentication, feedback, notices, and academic management. Relationships between collections are logically defined through common attributes like *email*, *departmentName*, and *className*, enabling seamless cross-referencing without the need for traditional foreign keys. The **Entity Relationship (ER) Diagram** represents how these collections interact to maintain academic workflows — from user registration and faculty assignment to feedback tracking and announcements. This structure ensures scalability, security, and real-time access to academic insights.

5.4.1 MCA Collection

This collection stores detailed information about students enrolled in the MCA program, including their email, roll number, and academic details such as course, semester, and class section. Refer to Table 5.1.

Table 5.1: MCA Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each student record
email	String	Student's official email ID
name	String	Student's full name
rollnumber	String	Unique roll number of the student
department	String	Name of the department (e.g., "Department of Computer Science (PG)")
course	String	Course name (e.g., "MCA")
semester	String	Current semester of the student
class	String	Class section (e.g., "1MCA-A")

5.4.2 departments Collection

This collection defines each department, including department type, class names, semester, and the list of subjects offered. Refer to Table 5.2.

Table 5.2: departments Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each department
className	String	Name of the class (e.g., “1MCA-B”)
departmentName	String	Name of the department
departmentType semester	String	Type of department (UG / PG)
semester	Integer	Current semester number
subjects	Array of Objects	List of subjects with course codes and names

5.4.3 faculty Collection

This collection contains the details of faculty members, including department, joining date, and faculty ID for internal mapping. Refer to Table 5.3.

Table 5.3: faculty Collection

Field Name	Data Type	Description
_id	ObjectID	Unique identifier for each faculty record
Department	String	Department name
DepartmentType	String	Department level (UG / PG)
Email	String	Faculty’s email address
facultyID	String	Unique faculty ID
fullName	String	Full name of the faculty
joiningDate	Date	Date of joining the institution

5.4.4 feedbacks Collection

This collection stores feedback submitted by students for specific subjects and faculty members. Each document captures rating-based and subjective feedback. Refer to Table 5.4.

Table 5.4: feedbacks Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each feedback entry
Subject	String	Name of the subject
CourseCode	String	Course code of the subject
Faculty	String	Faculty name for the course
studentName	String	Name of the student giving feedback
StudentEmail	String	Student's email ID
studentRoll	String / Null	Roll number of the student
Punctual	String	Yes/No for faculty punctuality
Clarity	String	Star rating for clarity of explanation
Engaging	String	Yes/No for interactivity
Pace	String	Feedback on pace (Slow, Fast, Normal)
Satisfaction	String	Overall satisfaction rating
comments	String	Additional feedback or remarks

5.4.5 student-login Collection

This collection stores authentication details for students, ensuring secure login with encrypted passwords and account verification tracking. Refer to Table 5.5.

Table 5.5: student-login Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each login record
Name	String	Student's name
Email	String	Student's email used for login
Password	String	Hashed password using bcrypt
CreatedAt	Date	Account creation date
isVerified	Boolean	Indicates if the account is verified
temporaryPassword	Boolean	Shows if a temporary password is used for login

5.4.5 subjects_assigned Collection

This collection defines which faculty members are assigned to specific classes and subjects. Refer to Table 5.6.

Table 5.6: subjects_assigned Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for the record
Clasname	String	Name of the class assigned
departmentName	String	Department name
departmentType	String	Department level (UG / PG)
faculty	String	Name of the assigned faculty
semester	Integer	Semester number
subjects	Array	List of subjects assigned to the faculty

6. SOURCE CODE

Home.component.html

```
<!-- hero.component.html -->
<section class="relative w-full min-h-screen overflow-hidden bg-white">

  <!-- Book Background - positioned to cover the page -->
  <div class="absolute inset-0 flex items-center justify-center">
    
  </div>

  <!-- Floating Graduation Caps - Left Side -->
  <div class="absolute left-8 top-20 animate-float-cap-1">
    <svg class="w-12 h-12 text-orange-400 opacity-60" viewBox="0 0 24 24"
      fill="currentColor">
      <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-4L12 15l-6.5-3.5z"/>
    </svg>
  </div>
```

```
<div class="absolute left-16 bottom-32 animate-float-cap-2">  
  <svg class="w-10 h-10 text-orange-500 opacity-50" viewBox="0 0 24 24"  
fill="currentColor">  
    <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-  
4L12 15l-6.5-3.5z"/>  
  </svg>  
</div>
```

```
<div class="absolute left-24 top-1/3 animate-float-cap-3">  
  <svg class="w-8 h-8 text-orange-300 opacity-70" viewBox="0 0 24 24"  
fill="currentColor">  
    <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-  
4L12 15l-6.5-3.5z"/>  
  </svg>  
</div>
```

```
<!-- Floating Graduation Caps - Right Side -->  
<div class="absolute right-12 top-24 animate-float-cap-4">  
  <svg class="w-11 h-11 text-orange-400 opacity-65" viewBox="0 0 24 24"  
fill="currentColor">  
    <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-  
4L12 15l-6.5-3.5z"/>  
  </svg>  
</div>
```

```
<div class="absolute right-20 bottom-40 animate-float-cap-5">
```

```

<svg class="w-9 h-9 text-orange-500 opacity-55" viewBox="0 0 24 24"
fill="currentColor">
  <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-
4L12 15l-6.5-3.5z"/>
</svg>
</div>

```

```

<div class="absolute right-8 top-1/2 animate-float-cap-6">
  <svg class="w-10 h-10 text-orange-300 opacity-60" viewBox="0 0 24 24"
fill="currentColor">
    <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zm-6.5 8.5v4l6.5 3.5 6.5-3.5v-
4L12 15l-6.5-3.5z"/>
  </svg>
</div>

```

```

<!-- Teacher - Left of Center (beside book spine) -->
<div class="absolute left-1/2 top-1/2 transform -translate-x-[280px] md:-
translate-x-[350px] lg:-translate-x-[420px] -translate-y-[55%] animate-float-
slow">
  
</div>

```

```
<!-- Student - Right of Center (beside book spine) -->
<div class="absolute left-1/2 top-1/2 transform scale-150 -translate-x-[260px]
md:translate-x-[110px] lg:translate-x-[180px] -translate-y-[180px] animate-float-
slow-delayed">
  
</div>

<!-- Main Content -->
<div class="relative z-10 flex flex-col items-center justify-center min-h-screen
px-4 py-16">

  <!-- Heading Section -->
  <div class="text-center mb-16 max-w-3xl">
    <h1 class="text-4xl md:text-5xl lg:text-6xl font-bold mb-6 text-gray-800">
      Welcome to <span class="text-orange-500">KJC VOX</span>
    </h1>
    <p class="text-xl md:text-2xl lg:text-3xl text-gray-600 font-medium">
      A Safe space for Students to share honest feedback
    </p>
  </div>

  <!-- Button at Bottom -->
```

```

<div class="mt-auto">
  <a href="/role-selection">
    <button
      class="px-8 py-4 md:px-10 md:py-5 bg-orange-500 hover:bg-orange-600
text-white text-lg md:text-xl font-bold rounded-full shadow-xl hover:shadow-2xl
transform hover:scale-105 transition-all duration-300"
    >
      Choose your role
    </button>
  </a>
</div>

</div>

</section>

```

Login.Component.html

```

<!-- Outer Wrapper -->
<div class="min-h-screen flex flex-col bg-gradient-to-br from-gray-50 to-gray-100">
  <!-- Header -->
  <header class="bg-white flex justify-between items-center px-6 py-4 shadow-sm sticky top-0 z-50">
    <div class="flex items-center gap-3">
      
      <span class="text-xl md:text-2xl font-bold text-gray-800">
        KJC <span class="text-yellow-500">VOX</span>

```

```

    </span>
  </div>

  <nav class="flex items-center gap-4">
    <ul class="hidden md:flex gap-4">
      <li><a href="/home" class="text-gray-700 font-medium hover:text-yellow-500 px-3 py-2
rounded transition-colors">Home</a></li>
      <li><a href="/register" class="text-gray-700 font-medium hover:text-yellow-500 px-3 py-2
rounded transition-colors">Register</a></li>
    </ul>

    <div class="hamburger md:hidden flex flex-col cursor-pointer" (click)="toggleMenu()">
      <div class="w-6 h-0.5 bg-gray-700 my-1 transition-all"></div>
      <div class="w-6 h-0.5 bg-gray-700 my-1 transition-all"></div>
      <div class="w-6 h-0.5 bg-gray-700 my-1 transition-all"></div>
    </div>
  </nav>

  <!-- Mobile Menu -->
  <ul [ngClass]="{ 'hidden': !isMobileMenuOpen }" class="md:hidden flex flex-col gap-2
absolute right-6 top-16 bg-white p-4 w-48 shadow-lg rounded-lg">
    <li><a href="/home" class="text-gray-700 font-medium hover:text-yellow-500 px-3 py-2
rounded transition-colors block" (click)="closeMobileMenu()">Home</a></li>
    <li><a href="/register" class="text-gray-700 font-medium hover:text-yellow-500 px-3 py-2
rounded transition-colors block" (click)="closeMobileMenu()">Register</a></li>
  </ul>
</header>

  <!-- Main Content -->
  <main class="flex-1 flex justify-center items-center px-5 py-12">
    <div class="w-full max-w-6xl">

```



```

<!-- Title Section -->
<div class="text-center mb-10">
  <h1 class="text-3xl md:text-4xl font-bold text-gray-800 mb-3">
    Login to <span class="text-yellow-500">Get Started</span>
  </h1>
  <p class="text-gray-600 text-sm md:text-base max-w-2xl mx-auto">
    Secure access for students, faculties & admins. Your data is safe and anonymous.
  </p>
</div>

<!-- Card Container -->
<div class="bg-white rounded-2xl shadow-xl overflow-hidden max-w-4xl mx-auto">
  <div class="flex flex-col md:flex-row">

    <!-- Left Side - Info Section -->
    <div class="md:w-2/5 bg-gradient-to-br from-blue-50 to-green-50 p-8 flex flex-col justify-
center items-center text-center">
      <div class="mb-6">
        
        <h2 class="text-2xl font-bold text-gray-800 mb-2">
          KJC <span class="text-yellow-500">VOX</span>
        </h2>
        <p class="text-sm text-gray-600 mb-4">Secure Login Portal</p>
      </div>

      <div class="space-y-4 text-left w-full max-w-xs">
        <div class="flex items-start gap-3 bg-white/60 p-3 rounded-lg backdrop-blur-sm
hover:bg-white hover:shadow-md transform hover:-translate-y-1 transition-all duration-300
cursor-pointer">
          <span class="text-2xl"><img alt="lock icon" /></span>

```

```

<div>
  <h3 class="font-semibold text-gray-800 text-sm">Secure Platform</h3>
  <p class="text-xs text-gray-600">Your data is encrypted and protected</p>
</div>
</div>

```

```

<div class="flex items-start gap-3 bg-white/60 p-3 rounded-lg backdrop-blur-sm
hover:bg-white hover:shadow-md transform hover:-translate-y-1 transition-all duration-300
cursor-pointer">

```

```

  <span class="text-2xl"><img alt="Feedback icon" data-bbox="388 331 408 346"/></span>
  <div>
    <h3 class="font-semibold text-gray-800 text-sm">Honest Feedback</h3>
    <p class="text-xs text-gray-600">Share your thoughts anonymously</p>
  </div>
</div>

```

```

<div class="flex items-start gap-3 bg-white/60 p-3 rounded-lg backdrop-blur-sm
hover:bg-white hover:shadow-md transform hover:-translate-y-1 transition-all duration-300
cursor-pointer">

```

```

  <span class="text-2xl"><img alt="Academic Growth icon" data-bbox="388 596 408 611"/></span>
  <div>
    <h3 class="font-semibold text-gray-800 text-sm">Academic Growth</h3>
    <p class="text-xs text-gray-600">Improve teaching and learning</p>
  </div>
</div>
</div>

```

```

<!-- Right Side - Form Section -->
<div class="md:w-3/5 bg-gradient-to-br from-slate-700 to-slate-800 p-8 md:p-10">
  <div class="max-w-md mx-auto">

```

```

<h2 class="text-2xl font-bold text-white mb-2">Welcome Back</h2>
<p class="text-gray-300 text-sm mb-6">Sign in to access your account</p>

<form #loginForm="ngForm" (ngSubmit)="submitLoginForm()" novalidate
class="space-y-5">
  <!-- Email Field -->
  <div>
    <label class="block text-sm font-medium text-gray-200 mb-2" for="email">
      Email Address
    </label>
    <div class="relative">
      <span class="absolute left-3 top-3 text-gray-400">✉ </span>
      <input
        type="email"
        id="email"
        name="email"
        [(ngModel)]="email"
        placeholder="name@kristujayanti.com"
        required
        #emailInput="ngModel"
        class="w-full pl-10 pr-4 py-3 rounded-lg border-2 border-transparent focus:border-
yellow-500 focus:outline-none text-gray-800 bg-white placeholder-gray-400 transition-all"
        [class.border-red-500]="emailInput.invalid && emailInput.touched"
      />
    </div>
    <div *ngIf="emailInput.invalid && emailInput.touched" class="text-sm text-red-300
mt-1">
      <span *ngIf="emailInput.errors?.['required']">Email is required</span>
      <span *ngIf="emailInput.errors?.['email']">Please enter a valid email</span>
    </div>
  </div>

```

```

<!-- Password Field -->
<div>
  <label class="block text-sm font-medium text-gray-200 mb-2" for="password">
    Password
  </label>
  <div class="relative">
    <span class="absolute left-3 top-3 text-gray-400">🔑 </span>
    <input
      type="password"
      id="password"
      name="password"
      [(ngModel)]="password"
      placeholder="Enter your password"
      required
      #passwordInput="ngModel"
      class="w-full pl-10 pr-4 py-3 rounded-lg border-2 border-transparent focus:border-
yellow-500 focus:outline-none text-gray-800 bg-white placeholder-gray-400 transition-all"
      [class.border-red-500]="passwordInput.invalid && passwordInput.touched"
    />
  </div>
  <div *ngIf="passwordInput.invalid && passwordInput.touched" class="text-sm text-
red-300 mt-1">
    <span *ngIf="passwordInput.errors?.['required']">Password is required</span>
  </div>
</div>

<!-- Forgot Password -->
<div class="text-right">
  <a href="/forgot-password" class="text-sm text-gray-300 hover:text-white underline
transition-colors">

```

```

    Forgot Password?
  </a>
</div>

<!-- Submit Button -->
<button
  type="submit"
  class="w-full bg-yellow-500 text-white font-bold py-3 rounded-lg hover:bg-yellow-
600 transition-all duration-300 shadow-lg hover:shadow-xl transform hover:-translate-y-0.5"
  [disabled]="isSubmitting || loginForm.invalid"
>
  <span *ngIf="!isSubmitting">🔑 Sign In</span>
  <span *ngIf="isSubmitting" class="flex items-center justify-center gap-2">
    <span class="animate-spin h-4 w-4 border-2 border-white border-t-transparent
rounded-full"></span>
    Signing in...
  </span>
</button>

<!-- Error Message -->
<div *ngIf="errorMessage" class="bg-red-500/20 border border-red-400 text-red-200
px-4 py-3 rounded-lg text-sm text-center">
  {{ errorMessage }}
</div>

<!-- Privacy Notice -->
<div class="flex items-center justify-center gap-2 pt-2">
  <span class="text-xl">🔒</span>
  <p class="text-xs text-gray-300">Your data is secure and anonymous</p>
</div>
</form>

```

```

    </div>
  </div>
</div>
</div>
</div>
</main>

<!-- Footer -->
<footer class="bg-slate-800 text-gray-300 text-center py-4 text-sm">
  © 2025 KJC VOX. All rights reserved.
</footer>
</div>

```

user-dash-board.component.html

```

<!-- Navbar -->
<header class="flex flex-wrap justify-between items-center px-8 py-3 bg-white border-b border-gray-300 shadow-sm">
  <div class="flex items-center gap-3">
    
    <h2 class="text-2xl font-bold">
      <span class="text-[#2e3d53]">KJC</span>
      <span class="text-[#f7b928]">VOX</span>
    </h2>
  </div>
  <div class="relative">
    

```

```

<div *ngIf="showLogout" class="absolute right-0 mt-2 w-40 bg-white rounded-lg shadow-lg
border border-gray-200 py-2 z-50">
  <button (click)="logout()" class="w-full text-left px-4 py-2 text-sm text-[#1d2d50]
hover:bg-gray-100 transition-colors flex items-center gap-2">
    <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
      <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M17 16l4-
4m0 0l-4-4m4 4H7m6 4v1a3 3 0 0 1-3 3H6a3 3 0 0 1-3 3V7a3 3 0 0 13-3h4a3 3 0 0 13
3v1"></path>
    </svg>
    Logout
  </button>
</div>
</div>
</header>

```

```

<!-- Blue section & white box reordered for mobile -->
<section class="bg-gradient-to-r from-[#2d4566] to-[#1d3a5c] text-white px-6 sm:px-10 py-10
rounded-b-2xl flex flex-col-reverse sm:flex-row gap-8 sm:gap-10 justify-between shadow-lg">
  <!-- Blue Greeting Box -->
  <div class="flex flex-col sm:flex-row items-center gap-6 flex-1">
    
    <div class="text-center sm:text-left">
      <h2 *ngIf="userName; else fallback" class="text-3xl font-semibold">Hello, {{ userName
}}</h2>
      <ng-template #fallback><h2 class="text-3xl font-semibold">Welcome!</h2></ng-
template>
      <p class="my-4 text-base">This is your personalized space on Vox to share feedback, track
your classroom experience, and contribute to academic improvement.</p>

```

```
<button class="px-5 py-2 border-2 border-white text-white rounded-lg font-semibold
hover:bg-white hover:text-[#2d4566] transition-all duration-300 hover:shadow-lg">Let your
voice be heard anonymously and meaningfully</button>
```

```
</div>
```

```
</div>
```

```
<!-- White Box (should appear first on mobile) -->
```

```
<div class="bg-[#cfdce6] text-[#1d2d50] p-6 rounded-2xl max-w-lg shadow-md flex-1
hover:shadow-xl transition-shadow duration-300">
```

```
<h4 class="text-lg font-bold text-center mb-5">Ready to share your thoughts?</h4>
```

```
<div class="flex justify-around flex-wrap gap-5">
```

```
<div class="text-center flex-1 hover:scale-105 transition-transform duration-300 cursor-
pointer">
```

```

```

```
<p>Start your<br />feedback<br />for today</p>
```

```
</div>
```

```
<div class="text-center flex-1 hover:scale-105 transition-transform duration-300 cursor-
pointer">
```

```

```

```
<p>Let your<br />emotions<br />be connected</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
<!-- Feature Cards & Image with Quote -->
```

```
<section class="flex flex-wrap justify-center sm:justify-between gap-8 p-10">
```

```
<div class="flex flex-col sm:flex-row flex-wrap gap-6 items-center sm:items-start justify-
center sm:justify-start max-w-full sm:max-w-[55%]">
```



```
<div class="bg-white border-4 border-[#cdd6e2] rounded-2xl w-[180px] p-6 text-center shadow-md cursor-pointer hover:shadow-xl hover:border-[#2d4566] hover:-translate-y-2 transition-all duration-300" (click)="goToMySubjects()">
```

```

```

```
<h4 class="text-[#1d2d50] text-base font-semibold">All Subjects</h4>
```

```
</div>
```

```
<div class="bg-white border-4 border-[#cdd6e2] rounded-2xl w-[180px] p-6 text-center shadow-md cursor-pointer hover:shadow-xl hover:border-[#2d4566] hover:-translate-y-2 transition-all duration-300">
```

```

```

```
<h4 class="text-[#1d2d50] text-base font-semibold">My Feedback</h4>
```

```
</div>
```

```
<div class="bg-white border-4 border-[#cdd6e2] rounded-2xl w-[180px] p-6 text-center shadow-md cursor-pointer hover:shadow-xl hover:border-[#2d4566] hover:-translate-y-2 transition-all duration-300">
```

```

```

```
<h4 class="text-[#1d2d50] text-base font-semibold">Report Issue</h4>
```

```
</div>
```

```
<!-- Motivational Quote Card - Wider 4th Card -->
```

```
<div class="bg-gradient-to-br from-[#2d4566] to-[#1d3a5c] border-4 border-[#cdd6e2] rounded-2xl w-[400px] p-6 shadow-md hover:shadow-xl hover:-translate-y-2 transition-all duration-300 flex flex-col justify-center items-center">
```

```
<svg class="h-10 w-10 text-[#f7b928] mb-3" fill="currentColor" viewBox="0 0 24 24">
```

```
<path d="M14.017 21v-7.391c0-5.704 3.731-9.57 8.983-10.609l.995 2.151c-2.432.917-3.995 3.638-3.995 5.849h4v10h-9.983zm-14.017 0v-7.391c0-5.704 3.748-9.57 9-10.609l.996 2.151c-2.433.917-3.996 3.638-3.996 5.849h3.983v10h-9.983z"/>
```

```
</svg>
```

```
<p class="text-white text-base font-medium leading-relaxed text-center mb-3 px-2">"Success is not final, failure is not fatal: it is the courage to continue that counts."</p>
```

```
<p class="text-[#f7b928] text-sm font-semibold">- Winston Churchill</p>
```

```

    </div>
  </div>

  <div class="w-full sm:w-[40%] sm:-ml-8">
    
  </div>
</section>

<!-- Announcements Section -->
<section class="flex flex-wrap gap-6 justify-between px-8 py-2">
  <div class="flex flex-col sm:flex-row items-center sm:items-start gap-5 flex-1 bg-white
  border-2 border-[#cdd6e2] p-5 rounded-xl shadow-md hover:shadow-lg transition-shadow
  duration-300">
    
    <div>
      <h3 class="text-lg font-semibold mb-3 text-[#1d2d50]">Announcements</h3>
      <ul class="list-none text-sm leading-7">
        <li class="hover:bg-gray-50 p-2 rounded transition-colors"><span class="mr-
1">🔔</span> Haven't submitted mood logs this week? <span class="text-red-600 font-
semibold">Don't forget to check in!</span></li>
        <li class="hover:bg-gray-50 p-2 rounded transition-colors"><span class="mr-
1">🔔</span> Semester feedback deadline is July 30, 2025. <span class="text-blue-600 font-
semibold">Submit before it closes!</span></li>
        <li class="hover:bg-gray-50 p-2 rounded transition-colors"><span class="mr-
1">🔔</span> Your feedback is 100% anonymous and valuable. <span class="font-
semibold">Let your voice be heard.</span></li>
      </ul>
    </div>
  </div>
</div>

```

```

<div class="flex flex-col sm:flex-row items-start gap-5 flex-1 bg-white border-2 border-
[#cdd6e2] p-5 rounded-xl shadow-md hover:shadow-lg transition-shadow duration-300">
  
  <div class="flex-1 min-h-[180px]">
    <h3 class="text-lg font-semibold mb-3 text-[#1d2d50]">Upcoming Assignments</h3>
    <div class="max-h-[140px] overflow-y-auto pr-2 custom-scrollbar">
      <ul class="list-none text-sm space-y-2">
        <li *ngFor="let notice of notices; let i = index" class="hover:bg-gray-50 p-2 rounded
transition-colors">
          <span class="mr-1 text-[#2d4566]">►</span>
          <strong class="text-[#1d2d50]">{{ notice.title }}</strong>:
          <span class="text-gray-700">{{ notice.description }}</span>
          <span class="ml-2 text-xs text-gray-500">[{{ notice.date }}]</span>
        </li>
      </ul>
    </div>
  </div>
</div>
</div>
</section>

<!-- Footer -->
<footer class="bg-[#1d2d50] text-white text-center text-sm py-4 mt-5">
  © KJC VOX. All rights reserved.
</footer>

<style>
@keyframes wave {
  0%, 100% { transform: rotate(0deg); }
  25% { transform: rotate(5deg); }

```

```
75% { transform: rotate(-5deg); }  
}  
.animate-wave {  
  animation: wave 2s ease-in-out infinite;  
}  
  
/* Custom Scrollbar */  
.custom-scrollbar::-webkit-scrollbar {  
  width: 6px;  
}  
.custom-scrollbar::-webkit-scrollbar-track {  
  background: #f1f1f1;  
  border-radius: 10px;  
}  
.custom-scrollbar::-webkit-scrollbar-thumb {  
  background: #2d4566;  
  border-radius: 10px;  
}  
.custom-scrollbar::-webkit-scrollbar-thumb:hover {  
  background: #1d2d50;  
}  
</style>
```

AuthController.java

```
package org.example.controller;  
  
import com.google.gson.Gson;  
import org.example.service.AuthService;  
  
import static spark.Spark.*;  
  
public class AuthController {
```

```
private static final Gson gson = new Gson();
private static final AuthService authService = new AuthService();

public static void routes() {
    post("/api/send-code", (req, res) -> {
        class RequestBody {
            String email;
            String name;
        }

        RequestBody body = gson.fromJson(req.body(), RequestBody.class);
        // Changed from sendVerificationCode to sendCode
        boolean success = authService.sendCode(body.email, body.name);

        res.type("application/json");
        return gson.toJson(new Response(success,
            success ? "Verification code sent" : "User already exists or error sending email"));
    });

    post("/api/register", (req, res) -> {
        class RequestBody {
            String email;
            String code;
            String name;
        }

        RequestBody body = gson.fromJson(req.body(), RequestBody.class);
        boolean success = authService.verifyAndRegister(body.email, body.code);

        res.type("application/json");
        return gson.toJson(new Response(success,
```

```
        success ? "Registration successful" : "Invalid code or expired"));
    });
}

static class Response {
    boolean success;
    String message;

    Response(boolean success, String message) {
        this.success = success;
        this.message = message;
    }
}
}
```

LoginServlet.java

```
package org.example.servlet;

import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

import org.example.repository.UserRepository;
import org.example.repository.FacultyRepository;
import org.example.model.User;
import org.example.model.Faculty;
import org.example.utils.PasswordUtil;
import com.google.gson.Gson;

import java.io.*;
import java.util.stream.Collectors;
```

```
@WebServlet("/api/login")
public class LoginServlet extends HttpServlet {
    private final UserRepository userRepo = new UserRepository();
    private final FacultyRepository facultyRepo = new FacultyRepository();
    private final Gson gson = new Gson();

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException
    {
        resp.setContentType("application/json");
        resp.setCharacterEncoding("UTF-8");

        try {
            String body = req.getReader().lines().collect(Collectors.joining());
            if (body == null || body.trim().isEmpty()) {
                sendError(resp, HttpServletResponse.SC_BAD_REQUEST, "Request body is
empty");
                return;
            }

            LoginRequest login = gson.fromJson(body, LoginRequest.class);

            if (login.getEmail() == null || login.getPassword() == null) {
                sendError(resp, HttpServletResponse.SC_BAD_REQUEST, "Email and password
required");
                return;
            }

            String email = login.getEmail().trim().toLowerCase();
            String password = login.getPassword().trim();
```

```
//  Admin login
if ("admin@kristujayanti.com".equals(email) && "admin".equals(password)) {
    LoginResponse response = new LoginResponse(
        true,
        "Admin login successful",
        true,
        email,
        "Admin",
        "admin"
    );
    resp.setStatus(HttpServletResponse.SC_OK);
    resp.getWriter().write(gson.toJson(response));
    return;
}

//  Faculty login
Faculty faculty = facultyRepo.findByEmail(email);
if (faculty != null) {
    if (!"Faculty@123".equals(password)) {
        sendError(resp, HttpServletResponse.SC_UNAUTHORIZED, "Invalid faculty
credentials");
        return;
    }

    LoginResponse response = new LoginResponse(
        true,
        "Faculty login successful",
        true,
        faculty.getEmail(),
        faculty.getFullName(),
```



```
        "faculty"
    );
    resp.setStatus(HttpServletResponse.SC_OK);
    resp.getWriter().write(gson.toJson(response));
    return;
}

// ☒ Student login
User user = userRepo.findByEmail(email);
if (user == null || !PasswordUtil.checkPassword(password, user.getPassword())) {
    sendError(resp, HttpServletResponse.SC_UNAUTHORIZED, "Invalid student
credentials");
    return;
}

HttpSession session = req.getSession(true);
session.setAttribute("userEmail", user.getEmail());
session.setAttribute("isVerified", user.isVerified());

LoginResponse response = new LoginResponse(
    true,
    "Student login successful",
    user.isVerified(),
    user.getEmail(),
    user.getName(),
    "student"
);

resp.setStatus(HttpServletResponse.SC_OK);
resp.getWriter().write(gson.toJson(response));
```

```
    } catch (Exception e) {  
        e.printStackTrace();  
        sendError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "Server  
error");  
    }  
}
```

```
private void sendError(HttpServletResponse resp, int status, String message) throws  
IOException {  
    resp.setStatus(status);  
    LoginResponse errorResponse = new LoginResponse(false, message, false, null, null, null);  
    resp.getWriter().write(gson.toJson(errorResponse));  
}
```

// Inner classes

```
static class LoginRequest {  
    private String email;  
    private String password;  
  
    public String getEmail() { return email; }  
    public String getPassword() { return password; }  
  
    public void setEmail(String email) { this.email = email; }  
    public void setPassword(String password) { this.password = password; }  
}
```

```
static class LoginResponse {  
    private final boolean success;  
    private final String message;  
    private final boolean isVerified;  
    private final String email;
```

```
private final String name;
private final String role;

public LoginResponse(boolean success, String message, boolean isVerified,
                     String email, String name, String role) {
    this.success = success;
    this.message = message;
    this.isVerified = isVerified;
    this.email = email;
    this.name = name;
    this.role = role;
}

public boolean isSuccess() { return success; }
public String getMessage() { return message; }
public boolean isVerified() { return isVerified; }
public String getEmail() { return email; }
public String getName() { return name; }
public String getRole() { return role; }
}
}
```

MongoUtil.java

```
package org.example.utils;

import com.mongodb.ConnectionString;
import com.mongodb.MongoClientSettings;
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoDatabase;
import org.bson.codecs.configuration.CodecRegistry;
```

```
import org.bson.codecs.pojo.PojoCodecProvider;
import org.bson.codecs.configuration.CodecProvider;

import static org.bson.codecs.configuration.CodecRegistries.fromProviders;
import static org.bson.codecs.configuration.CodecRegistries.fromRegistries;

public class MongoUtil {
    private static final String CONNECTION_STRING =
"mongodb+srv://24mcab07:RadhaBhaskar%401@kjc-vox-
cluster.ps4pwaz.mongodb.net/kjc_vox_portal?retryWrites=true&w=majority&appName=KJC-
VOX-Cluster";
    private static final String DB_NAME = "kjc_vox_portal";
    private static MongoClient client;
    private static MongoDB database;

    static {
        try {
            // Create custom codec provider
            CodecProvider pojoCodecProvider = PojoCodecProvider.builder()
                .register("org.example.model")
                .automatic(true) // <-- required
                .build();

            // Combine with default codec registry
            CodecRegistry codecRegistry = fromRegistries(
                MongoClientSettings.getDefaultCodecRegistry(),
                fromProviders(pojoCodecProvider)
            );

            MongoClientSettings settings = MongoClientSettings.builder()
                .applyConnectionString(new ConnectionString(CONNECTION_STRING))
```

```
.codecRegistry(codecRegistry)
    .build();

client = MongoClient.create(settings);
database = client.getDatabase(DB_NAME);

System.out.println("MongoDB connection established successfully");

} catch (Exception e) {
    System.err.println("Failed to initialize MongoDB connection: " + e.getMessage());
    e.printStackTrace();
    throw new RuntimeException("Failed to initialize MongoDB connection", e);
}
}

public static MongoDBDatabase getDatabase() {
    if (database == null) {
        throw new RuntimeException("Database not initialized. Check MongoDB connection.");
    }
    return database;
}

public static void close() {
    if (client != null) {
        client.close();
    }
}
}

FeedbackRepository.java
package org.example.repository;
```

```
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import org.bson.Document;
import org.example.model.Feedback;
import org.example.utils.MongoUtil;

public class FeedbackRepository {
    private final MongoCollection<Document> collection;

    public FeedbackRepository() {
        MongoDatabase database = MongoUtil.getDatabase();
        this.collection = database.getCollection("feedbacks");
    }

    public void saveFeedback(Feedback feedback) {
        Document doc = new Document()
            .append("subject", feedback.getSubject())
            .append("courseCode", feedback.getCourseCode())
            .append("faculty", feedback.getFaculty())
            .append("studentName", feedback.getStudentName())
            .append("studentEmail", feedback.getStudentEmail())
            .append("studentRoll", feedback.getStudentRoll())
            .append("punctual", feedback.getPunctual())
            .append("clarity", feedback.getClarity())
            .append("engaging", feedback.getEngaging())
            .append("pace", feedback.getPace())
            .append("satisfaction", feedback.getSatisfaction())
            .append("comments", feedback.getComments());
        collection.insertOne(doc);
    }
}
```

FeedbackService.java

```
package org.example.service;

import org.example.model.Feedback;
import org.example.repository.FeedbackRepository;

public class FeedbackService {
    private final FeedbackRepository repository = new FeedbackRepository();

    public void submitFeedback(Feedback feedback) {
        repository.saveFeedback(feedback);
    }
}
```

Feedback.java

```
package org.example.model;

public class Feedback {
    private String subject;
    private String courseCode;
    private String faculty;
    private String studentName;
    private String studentEmail;
    private String studentRoll;
    private String punctual;
    private int clarity;
    private String engaging;
    private String pace;
    private int satisfaction;
```

```
private String comments;
```

```
// Getters and setters
```

```
public String getSubject() { return subject; }
```

```
public void setSubject(String subject) { this.subject = subject; }
```

```
public String getCourseCode() { return courseCode; }
```

```
public void setCourseCode(String courseCode) { this.courseCode = courseCode; }
```

```
public String getFaculty() { return faculty; }
```

```
public void setFaculty(String faculty) { this.faculty = faculty; }
```

```
public String getStudentName() { return studentName; }
```

```
public void setStudentName(String studentName) { this.studentName = studentName; }
```

```
public String getStudentEmail() { return studentEmail; }
```

```
public void setStudentEmail(String studentEmail) { this.studentEmail = studentEmail; }
```

```
public String getStudentRoll() { return studentRoll; }
```

```
public void setStudentRoll(String studentRoll) { this.studentRoll = studentRoll; }
```

```
public String getPunctual() { return punctual; }
```

```
public void setPunctual(String punctual) { this.punctual = punctual; }
```

```
public int getClarity() { return clarity; }
```

```
public void setClarity(int clarity) { this.clarity = clarity; }
```

```
public String getEngaging() { return engaging; }
```

```
public void setEngaging(String engaging) { this.engaging = engaging; }
```

```
public String getPace() { return pace; }
```



```
public void setPace(String pace) { this.pace = pace; }

public int getSatisfaction() { return satisfaction; }
public void setSatisfaction(int satisfaction) { this.satisfaction = satisfaction; }

public String getComments() { return comments; }
public void setComments(String comments) { this.comments = comments; }
}
```

FeedbackSummaryController.java

```
package org.example.controller;

import com.google.gson.Gson;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.model.Filters;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.bson.Document;
import org.example.utils.MongoUtil;

import java.io.IOException;
import java.util.*;

@WebServlet("/api/feedback-summary")
public class FeedbackSummaryController extends HttpServlet {

    private final MongoCollection<Document> collection =
```

```
MongoUtil.getDatabase().getCollection("feedbacks");
private final Gson gson = new Gson();

@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException
{
    String subject = req.getParameter("subject");
    String faculty = req.getParameter("faculty");

    if (subject == null || faculty == null) {
        resp.setStatus(HttpServletResponse.SC_BAD_REQUEST);
        resp.getWriter().write("{\"error\": \"Missing parameters\"}");
        return;
    }

    List<Document> feedbacks = collection.find(
        Filters.and(
            Filters.eq("subject", subject),
            Filters.eq("faculty", faculty)
        )
    ).into(new ArrayList<>());

    int total = feedbacks.size();
    int punctualYes = 0;
    int engagingYes = 0;
    double claritySum = 0;
    int normal = 0, fast = 0, slow = 0;
    List<String> comments = new ArrayList<>();

    for (Document f : feedbacks) {
        if ("yes".equalsIgnoreCase(f.getString("punctual"))) punctualYes++;
    }
}
```

```
if ("yes".equalsIgnoreCase(f.getString("engaging"))) engagingYes++;

Object clarityObj = f.get("clarity");
if (clarityObj instanceof Number) {
    claritySum += ((Number) clarityObj).doubleValue();
}

String pace = f.getString("pace");
if ("normal".equalsIgnoreCase(pace)) normal++;
else if ("fast".equalsIgnoreCase(pace)) fast++;
else if ("slow".equalsIgnoreCase(pace)) slow++;

String comment = f.getString("comments");
if (comment != null && !comment.trim().isEmpty()) {
    comments.add(comment);
}
}

Map<String, Object> paceMap = new HashMap<>();
paceMap.put("normal", normal);
paceMap.put("fast", fast);
paceMap.put("slow", slow);

Map<String, Object> result = new HashMap<>();
result.put("punctuality", total > 0 ? (punctualYes * 100 / total) + "%" : "0%");
result.put("clarity", total > 0 ? String.format("%.1f", claritySum / total) + "%/5" : "0/5");
result.put("engagement", total > 0 ? (engagingYes * 100 / total) + "%" : "0%");
result.put("pace", paceMap);
result.put("comments", comments);
resp.setContentType("application/json");
resp.getWriter().write(gson.toJson(result));  }}
```

7. SYSTEM TESTING

System testing is a critical phase in the development of the KJC Vox Portal, ensuring that all integrated modules — including user registration, authentication, feedback submission, faculty and course management, and notice handling — function together seamlessly as a unified system. The primary objective of this testing phase is to validate that the entire application performs as intended under realistic usage conditions. It verifies data integrity across MongoDB collections, the responsiveness of the frontend interface, and the proper interaction between user modules and backend APIs.

Through this process, issues such as incorrect data flow, validation errors, and integration bugs are identified and corrected. The successful execution of this testing confirms that KJC Vox Portal delivers a secure, consistent, and user-friendly experience to all stakeholders — students, faculty, and administrators.

7.1 Types of Testing

This section outlines the different types of testing performed on the KJC Vox system. Each testing category focused on validating specific modules and their interactions.

7.1.1 Validation Testing

Validation testing was carried out to ensure that all forms in the system — including registration, login, and feedback submission — correctly validate user input before processing.

Incorrect data formats, missing fields, and duplicate records were properly handled with meaningful error messages.

For example, during registration, invalid emails or short passwords triggered validation alerts, and existing email entries were restricted.

Refer to Appendix 10.1 – 10.2 for screenshots of form validation results.

7.1.2 Functional Testing

Functional testing verified that each core feature of the KJC Vox system performed according to the specified requirements.

This included testing login/logout, OTP verification, anonymous feedback submission, and admin-level CRUD operations for subjects and users.

Role-based access control (Student, Faculty, and Admin) was validated to ensure that users only accessed their respective modules.

Refer to Appendix 10.3 for related screenshots.

7.1.3 Integration Testing

Integration testing focused on data flow across interconnected modules — ensuring smooth communication between the frontend, backend APIs, and MongoDB collections.

This testing validated that updates made by the admin (e.g., assigning faculty or adding notices) reflected instantly on student dashboards, confirming successful synchronization between components.

Refer to Appendix 10.4 -10.8 for student dashboards and adding notices screenshots.

7.1.4 Security Testing

Security testing was conducted to ensure the confidentiality, integrity, and authenticity of system data. This included verifying password encryption using bcrypt, JWT-based authentication tokens, and secure API access control for role-based permissions.

Unauthorized access attempts were blocked successfully.

7.1.5 Performance Testing

Performance testing measured the system's speed and responsiveness.

API response times for key modules such as login, feedback submission, and dashboard loading were monitored.

The dashboard loaded in under 2 seconds, even under concurrent access, proving that the system performs efficiently in real-world conditions.

7.2 Test Cases

The following tables summarize the major test cases executed during validation, functional, and integration testing phases.

These tests ensure that every critical function of the KJC Vox Feedback System operates as expected.

Test Case Table 7.1 – User Registration Form

Test ID	Test Condition	Test Description	Test Data	Actual Result	Expected Result	Final Result
TC-R-01	Name validation	User enters invalid characters in name	A!b1	System rejects invalid input	Error: “Invalid name format”	Pass
TC-R-02	Empty field check	User submits form without entering name	(blank)	System prompts for name	“Name is required”	Pass
TC-R-03	Valid registration	User enters valid name and details	Abi Alluri	Registration successful	Redirect to verification page	Pass
TC-R-04	Email format validation	User enters incorrect email	abialluri@	System rejects email	“Invalid email format”	Pass
TC-R-05	Duplicate email check	User uses existing registered email	24mcab07@gmail.com	Duplicate found	“Email already exists”	Pass
TC-R-06	Password length validation	Password less than 8 chars	12345	System rejects password	“Password too short”	Pass

TC-R-07	Password encryption	Verify stored password hash	12345678	Encrypted hash saved	Password stored securely	Pass
TC-R-08	Verification flag	New user not verified yet	False	isVerified = false	User must verify before login	Pass

Test Case Table 7.2 – Login Form

Test ID	Test Condition	Test Description	Test Data	Actual Result	Expected Result	Final Result
TC-L-01	Email field validation	Email left empty	(blank)	System shows warning	“Email cannot be empty”	Pass
TC-L-02	Password field validation	Password left empty	(blank)	System shows warning	“Password cannot be empty”	Pass
TC-L-03	Invalid credentials	Wrong password entered	24mcab07@ Kristujayanti.com (Wrong pass)	Login rejected	“Invalid credentials”	Pass
TC-L-04	Unregistered email	Non-existent email used	24mcab97@ Kristujayanti.com	Record not found	“User not found”	Pass
TC-L-05	Valid login	Correct credentials entered	24mcab07@ Kristujayanti.com (correct pass)	Logged in successfully	Redirect to student dashboard	Pass
TC-L-06	Token validation	JWT token verified	Valid token	Access granted	Secure access confirmed	Pass

Test Case Table 7.3 – Feedback Form

Test ID	Test Condition	Test Description	Test Data	Actual Result	Expected Result	Final Result
TC-F-01	Subject field validation	Subject not selected	(blank)	Error displayed	“Please select subject”	Pass
TC-F-02	Faculty mapping check	Selected subject linked with faculty	dsa → MOKSHITH	Correct faculty fetched	Faculty matched	Pass
TC-F-03	Feedback rating input	User selects stars for clarity and satisfaction	4, 5	Stored successfully	Ratings saved correctly	Pass
TC-F-04	Text input validation	User leaves comments blank	(blank)	Optional field handled	No error	Pass
TC-F-05	Feedback storage	Verify database insert operation	Valid feedback form	Saved in feedbacks collection	Document inserted	Pass
TC-F-06	Anonymous integrity	Check anonymity handling	Feedback without visible name	Student details masked	Feedback stored anonymously	Pass

Test Case Table 7.4 – Dashboard & Notice Module

Test ID	Test Condition	Test Description	Test Data	Actual Result	Expected Result	Final Result
TC-D-01	Notice retrieval	Load all active notices	Valid user login	Notices displayed	Announcements visible	pass
TC-D-02	Assignment visibility	Filter by noticeType = assignment	Student login	Assignment shown	Filter works correctly	pass
TC-D-03	Subject listing	Display subjects by department and semester	1MCA-B	All subjects listed	Data retrieved from departments	pass
TC-D-04	Assigned faculty	Fetch subject-faculty mapping	Python → Femi F	Faculty name displayed	Correctly matched	pass
TC-D-05	Dashboard load time	Measure API response	2 sec avg	Dashboard loads instantly	Acceptable response time	pass

Test Case Table 7.5 – Data Consistency & Sync

Test ID	Test Condition	Test Description	Test Data	Actual Result	Expected Result	Final Result
TC-A-01	Faculty assignment sync	Add new subject-faculty mapping	Angular → Dr. Femi	Updated instantly	Displayed in student dashboard	pass
TC-A-02	Feedback retrieval	Faculty views received feedback	Query by courseCode = cs013	Records fetched	Correct entries shown	pass
TC-A-03	Department data update	New subject added to department	“AI Tools”	Reflected immediately	Data retrieved from Appears under 1MCA-B	pass
TC-A-04	Assigned faculty	Admin adds new assignment	“Python Mini Project”	Notice displayed instantly	Visible to students	pass
TC-A-05	Data persistence check	MongoDB record consistency	Refresh app	No data loss	Data remains accurate	pass

8. SYSTEM IMPLEMENTATION

8.1 System Deployment

The deployment phase of the **KJC Vox Feedback System** involves hosting and configuring the application in a live environment, ensuring it can be securely accessed by students, faculty, and administrators. The goal of deployment is to deliver a fully operational, reliable, and scalable feedback management system accessible across browsers and devices with minimal latency and maximum data security.

Since **KJC Vox** is a **web-based platform**, users can access it directly through any modern web browser without the need for separate installations. The system is deployed on both **cloud and local environments** for testing and performance validation. The deployment ensures smooth integration between the Angular frontend, Java-based backend, and MongoDB Atlas database, while maintaining secure communication through encrypted connections and authentication mechanisms.

8.1.1 Deployment Steps

1. **Server Configuration:** The backend system, developed using Java (Spring Boot) and hosted on a web server such as Apache Tomcat or AWS EC2, manages all application logic and APIs. The MongoDB Atlas cloud database is securely connected to the backend through environment variables for protected credential access. The backend handles OTP verification, feedback storage, and user authentication.
2. **Frontend Deployment:** The Angular frontend was built using the production command:
`ng build --prod`
The optimized build files were then deployed on web hosting services such as Netlify, Vercel, or Firebase Hosting. This setup ensures fast loading times, responsive design,

and reliable accessibility across devices.

3. **AI and API Integration:** The JavaMail API was configured in the backend for sending OTPs and password reset emails. All sensitive credentials such as SMTP keys, API tokens, and database URIs are stored securely in the environment configuration (application.properties or .env) files. Communication between frontend and backend occurs over HTTPS using RESTful APIs for secure and authenticated transactions. **Database Deployment:** The MySQL database was deployed on a remote or cloud-based server (e.g., AWS RDS, Google Cloud SQL) to enable scalability and data security. Regular backups were scheduled for data protection.
4. **Testing the Live Environment:** The application uses MongoDB Atlas, a cloud-based NoSQL database service, for storing user, feedback, and department data. Collections such as Student-login, MCA, feedbacks, and faculty are deployed on Atlas clusters with real-time monitoring and automatic backup configurations to ensure data integrity and recovery.
5. **Maintenance and Monitoring:** After deployment, the system was tested in the live environment for:
 - API response time
 - Cross-browser compatibility
 - OTP delivery success rate
 - Feedback form submission and retrieval
 - Role-based access verification

Testing confirmed that all modules worked as intended with smooth data synchronization between the frontend, backend, and cloud database.

6. **Maintenance and Monitoring :** Once deployed, continuous monitoring tools (such as MongoDB Atlas Monitor and Spring Boot Actuator) were integrated to track database performance, user activity, and server uptime. Regular updates, backups, and security audits are performed to maintain data protection, improve performance, and ensure the system's long-term stability.

8.2 Post-Deployment Evaluation

After the system was successfully deployed, continuous evaluation was carried out to measure its performance, reliability, and user satisfaction. The live testing involved active participation from students, faculty, and administrators, allowing real-world validation of functionalities such as feedback submission, dashboard visualization, and admin management.

Key performance indicators (KPIs) — such as **response latency**, **data retrieval accuracy**, and **email delivery rate** — were monitored to ensure optimal operation. User feedback collected during this phase helped identify areas for improvement, such as faster form submissions, better notice visibility, and enhanced analytics visualization for faculty and admin dashboards.

Regular updates are scheduled to keep the system compatible with the latest versions of **Angular**, **Java**, and **MongoDB Atlas**. Automatic backups and cloud recovery mechanisms safeguard the system against unexpected failures.

The successful deployment of **KJC Vox Feedback System** marks a significant advancement toward promoting a transparent, anonymous, and data-driven feedback culture in academic environments.

By integrating secure technologies, modern frameworks, and scalable cloud infrastructure, the system ensures reliability, privacy, and long-term sustainability for institutional use.

9 CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The KJC Vox Feedback System was conceptualized and developed to provide a secure, transparent, and anonymous platform for students to share feedback about their faculty and courses. The primary objective of this system is to encourage honest communication and continuous improvement within academic institutions while ensuring complete confidentiality of student identities.

Unlike traditional feedback systems that rely on manual paper forms or insecure online surveys, KJC Vox offers a structured, automated, and cloud-integrated solution that promotes data privacy, authenticity, and ease of management.

The system has been designed with a three-tier architecture, incorporating Angular for the front end, Java (Spring Boot) for the backend, and MongoDB Atlas for cloud-based data storage — ensuring high scalability, reliability, and performance.

Through role-based access, students can securely log in, verify their details, and submit feedback anonymously; faculty members can view aggregated results and analyze performance metrics; and administrators can manage departments, subjects, faculty assignments, and announcements efficiently through an interactive dashboard.

The integration of JavaMail API further enhances system security by facilitating OTP-based verification and password recovery.

The successful implementation and testing of KJC Vox validate that it fulfills its core objectives:

- To collect student feedback anonymously and securely.
- To provide faculty members with summarized insights and analytical reports for self-assessment.
- To enable administrators to manage and monitor institutional feedback through a centralized system.

The KJC Vox Feedback System thus represents a significant step forward in enhancing communication and transparency between students and academic management.

By leveraging modern web technologies and cloud infrastructure, it transforms traditional feedback collection into a data-driven, efficient, and secure digital process, helping institutions improve teaching quality and overall student experience.

9.2 Future Enhancement

While the current version of KJC Vox effectively achieves its intended objectives, there are several areas where the system can be expanded and enhanced to provide greater functionality, accessibility, and analytical depth in the future.

1. AI-Based Sentiment Analysis:

Future versions can integrate natural language processing (NLP) techniques to analyze the sentiment of student comments automatically. This would help identify trends in satisfaction levels and detect areas that need attention.

2. Data Visualization Dashboard:

Advanced data visualization tools can be added to the admin and faculty dashboards to provide real-time insights into feedback trends, ratings distribution, and performance graphs.

3. Mobile Application Development:

Building an Android and iOS version of the KJC Vox system will enhance accessibility, allowing students and faculty to interact with the system conveniently on their mobile devices.

4. Automated Notifications and Alerts:

Implementing push or email notifications for new announcements, deadlines, or feedback summaries will keep all users informed in real time.

5. Integration with Academic Portals (LMS):

Future iterations can integrate the feedback system with existing college Learning Management Systems (LMS) for seamless synchronization of student and faculty data.

6. Predictive Analytics for Performance Improvement:

Using AI-driven predictive analytics, the system can forecast performance patterns and suggest targeted improvement strategies for departments or faculty.

Multi-Department and Cross-Course Expansion:

The system can be scaled to support multiple courses and departments simultaneously across different programs (UG & PG), creating a unified institutional feedback platform.

7. Enhanced Privacy through Blockchain (Optional Exploration):

For institutions prioritizing transparency and immutability, blockchain technology can be explored to ensure that submitted feedback remains tamper-proof and fully anonymous.

8. Offline Feedback Support:

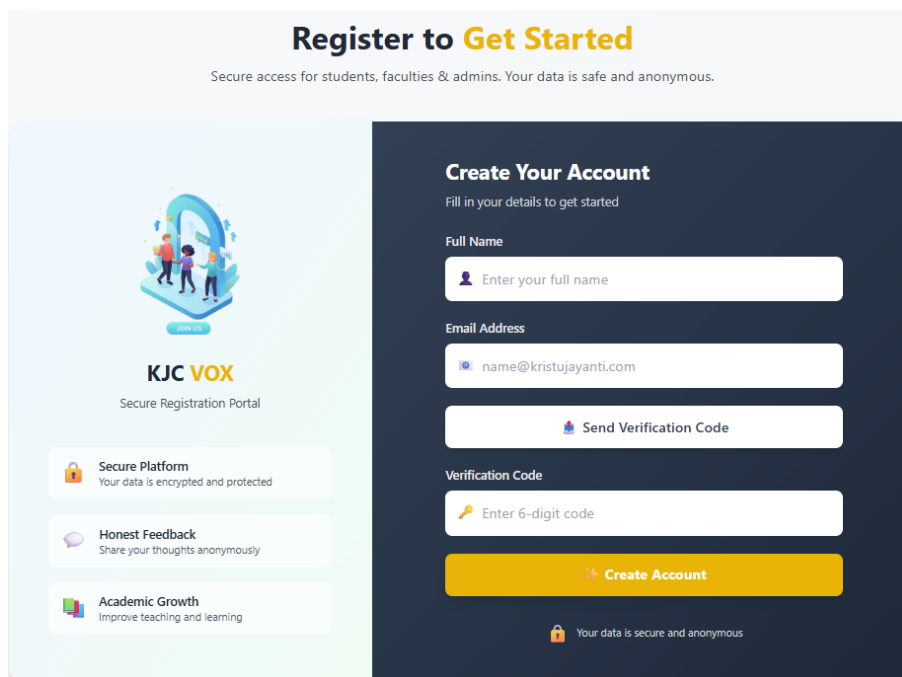
Providing an offline submission mode that syncs data once connected to the internet will help institutions with limited connectivity maintain consistent feedback collection.

The KJC Vox Feedback System sets a foundation for modern, ethical, and data-driven academic feedback collection. By combining usability, anonymity, and analytical intelligence, it ensures that every student's voice contributes to institutional growth and teaching excellence.

With continued innovation — including AI integration, analytics, and mobile accessibility — KJC Vox has the potential to evolve into a comprehensive digital platform that empowers academic institutions to make informed, transparent, and impactful decisions.

10 APPENDICES

10.1 Screenshots



The screenshot shows the 'Register to Get Started' page for KJC VOX. The header includes the title and a security note. The left sidebar features the KJC VOX logo and three benefits: Secure Platform, Honest Feedback, and Academic Growth. The main form area is titled 'Create Your Account' and contains fields for Full Name, Email Address, and Verification Code, along with a 'Send Verification Code' button and a 'Create Account' button. A security note is at the bottom.

Register to Get Started
Secure access for students, faculties & admins. Your data is safe and anonymous.

Create Your Account
Fill in your details to get started

Full Name
Enter your full name

Email Address
name@kristujayanti.com

Send Verification Code

Verification Code
Enter 6-digit code

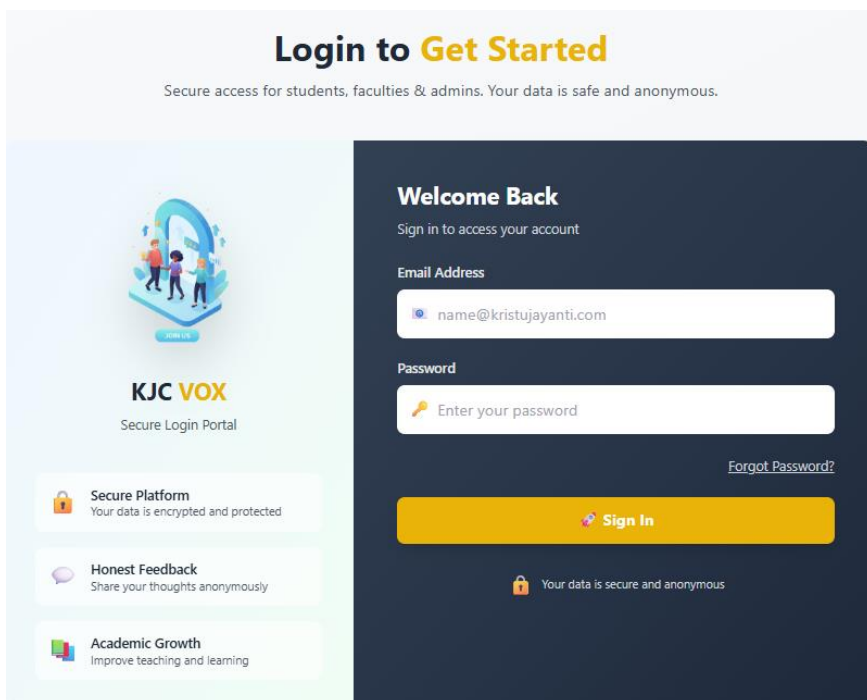
Create Account

Your data is secure and anonymous

KJC VOX
Secure Registration Portal

- Secure Platform**
Your data is encrypted and protected
- Honest Feedback**
Share your thoughts anonymously
- Academic Growth**
Improve teaching and learning

Figure 10.1: Registration Page



The screenshot shows the 'Login to Get Started' page for KJC VOX. The header includes the title and a security note. The left sidebar features the KJC VOX logo and three benefits: Secure Platform, Honest Feedback, and Academic Growth. The main form area is titled 'Welcome Back' and contains fields for Email Address and Password, along with a 'Forgot Password?' link and a 'Sign In' button. A security note is at the bottom.

Login to Get Started
Secure access for students, faculties & admins. Your data is safe and anonymous.

Welcome Back
Sign in to access your account

Email Address
name@kristujayanti.com

Password
Enter your password

[Forgot Password?](#)

Sign In

Your data is secure and anonymous

KJC VOX
Secure Login Portal

- Secure Platform**
Your data is encrypted and protected
- Honest Feedback**
Share your thoughts anonymously
- Academic Growth**
Improve teaching and learning

Figure 10.2: Login Page

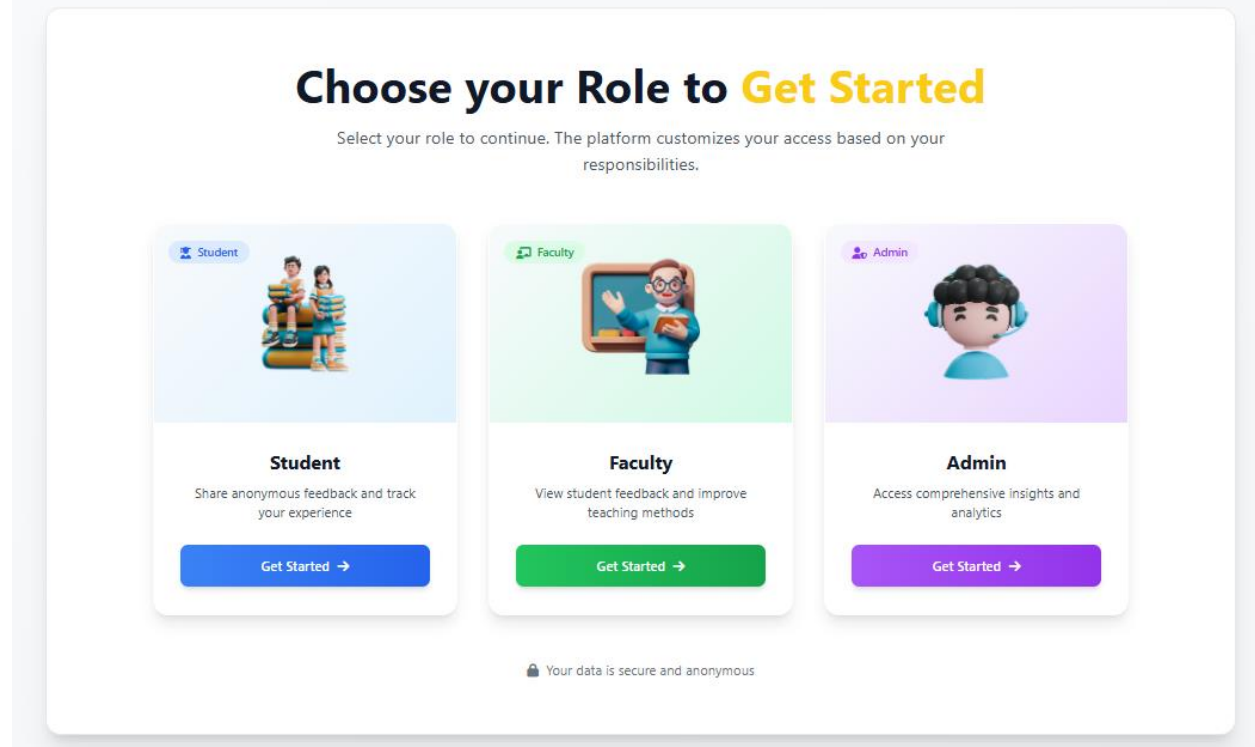


Figure 10.3: Role Selection Page

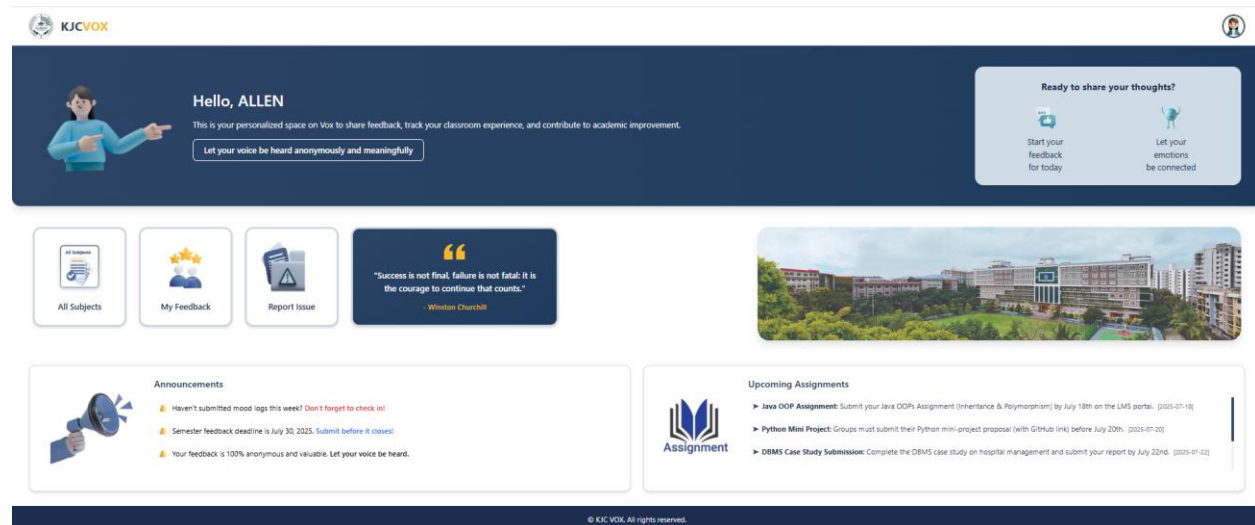
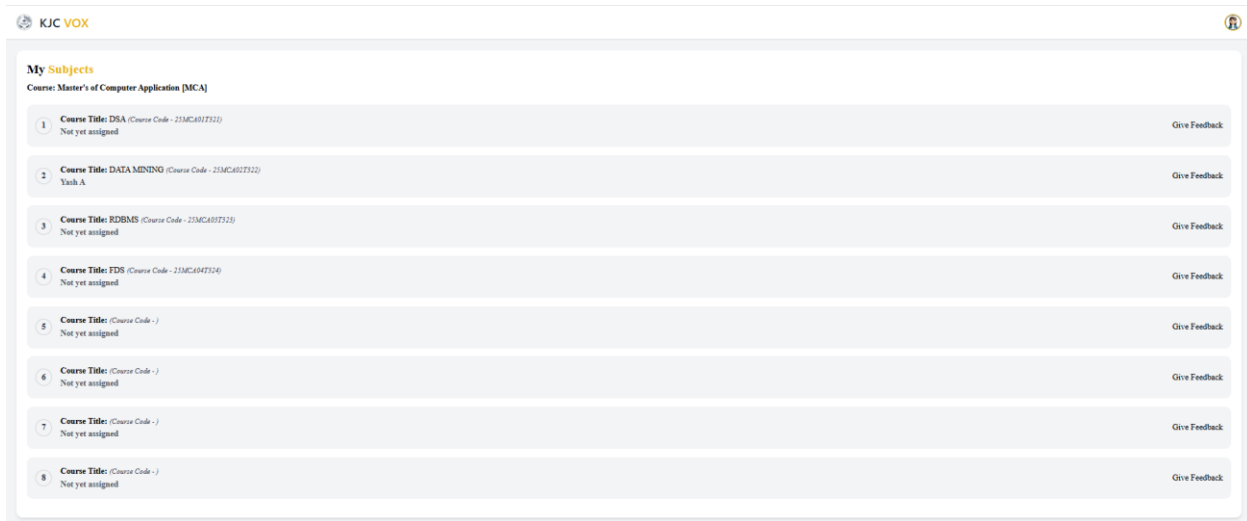


Figure 10.4: Student Dashboard Page



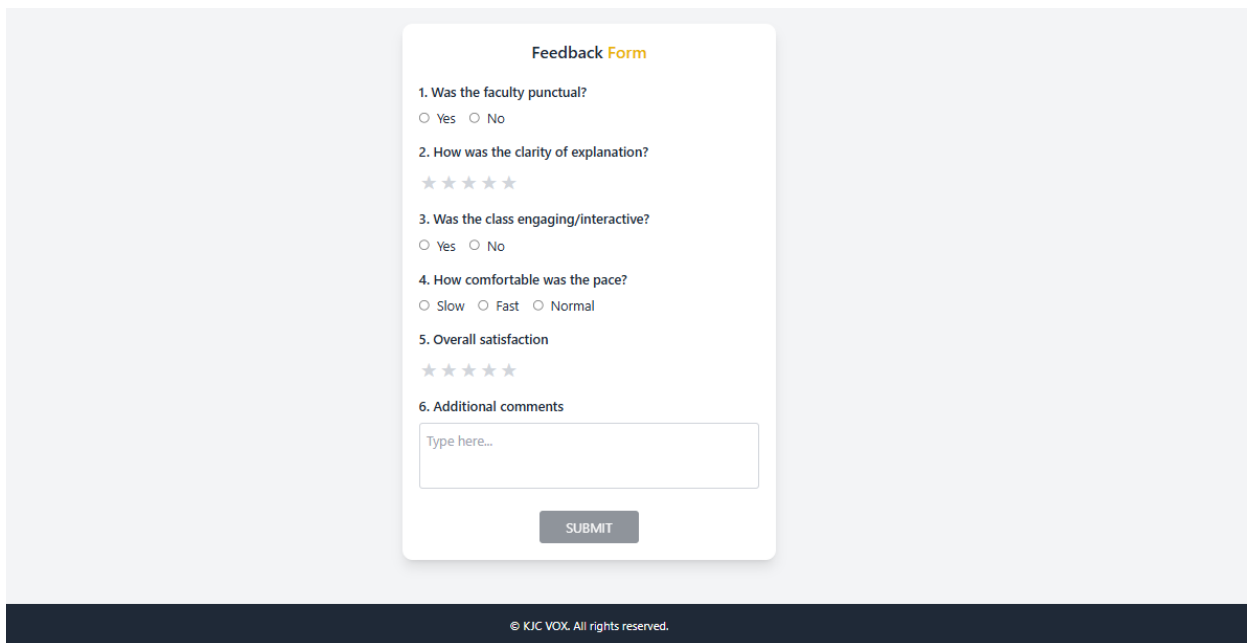
KJC VOX

My Subjects

Course: Master's of Computer Application [MCA]

1	Course Title: DSA (Course Code - 21MCA017121) Not yet assigned	Give Feedback
2	Course Title: DATA MINING (Course Code - 21MCA017122) Yash A	Give Feedback
3	Course Title: RDBMS (Course Code - 21MCA017123) Not yet assigned	Give Feedback
4	Course Title: FDS (Course Code - 21MCA047124) Not yet assigned	Give Feedback
5	Course Title: (Course Code -) Not yet assigned	Give Feedback
6	Course Title: (Course Code -) Not yet assigned	Give Feedback
7	Course Title: (Course Code -) Not yet assigned	Give Feedback
8	Course Title: (Course Code -) Not yet assigned	Give Feedback

Figure 10.5: Student Course view page



Feedback Form

- Was the faculty punctual?
☐ Yes ☐ No
- How was the clarity of explanation?
☐ ★ ☐ ★ ☐ ★ ☐ ★ ☐ ★
- Was the class engaging/interactive?
☐ Yes ☐ No
- How comfortable was the pace?
☐ Slow ☐ Fast ☐ Normal
- Overall satisfaction
☐ ★ ☐ ★ ☐ ★ ☐ ★ ☐ ★
- Additional comments
 Type here...

SUBMIT

© KJC VOX. All rights reserved.

Figure 10.6: Feedback form Page

Add Department

Department Name
Department of Computer Science

Department Type
PG

Class

Semester
I

Subjects

SUB1

Name

Course Code

SUB2

Name

Course Code

SUB3

Name

Course Code

SUB4

Name

Course Code

SUB5

Name

Course Code

SUB6

Name

Course Code

SUB7

Name

Course Code

SUB8

Name

Course Code

Submit

Figure 10.7: Add Department Page

Notices

Title of Notice
Assignment Deadline Extended

Date
dd-mm-yyyy

Notice For
Student/Faculty/Both

Notice Type
Academic/event/exam/general/urgent
Academic
Event
Exam
General
Urgent

SUBMIT

KJC VOX. All rights reserved.

Figure 10.8: Add Notice Page

REFERENCES

1. Angular Official Documentation — <https://angular.io/docs>
2. Spring Boot Framework Documentation — <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
3. MongoDB Atlas Cloud Database Documentation
<https://www.mongodb.com/docs/atlas/>
4. JavaMail API (Jakarta Mail) – Email Integration Guide —
<https://eclipse-ee4j.github.io/mail/>
5. JWT (JSON Web Token) Authentication Documentation —
<https://jwt.io/introduction>
6. Postman API Testing Tool — <https://www.postman.com>
7. MDN Web Docs – RESTful API Design Principles — https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Introduction
8. W3Schools – HTML, CSS, and JavaScript Fundamentals — <https://www.w3schools.com>
9. GitHub – Version Control and CI/CD Integration — <https://github.com>
10. Baeldung – Spring Boot & MongoDB Integration Tutorials
<https://www.baeldung.com/spring-boot-mongodb>